

Ausarbeitung: Stock-Tracker

Gruppenteilnehmer:

Bleron Beqiri – *****
Dominik Peschke – *****

Zugangsdaten Projekt

Link: <https://github.com/ArtificialDetour/Projektarbeit-Stock-Tracker>

Git-Projektzugang per SSH: `git@github.com:ArtificialDetour/Projektarbeit-Stock-Tracker.git`

Passwort NeonDB: ***** -> auf Papierabgabe!

WICHTIG: Das Passwort in der `application-dev.yaml` in Zeile 16 einfügen, sonst ist die Neon-Datenbank nicht nutzbar!

```
25 # Development profile settings.
24 spring:
23   # Docker Compose is disabled for cloud database development.
22   docker:
21     compose:
20       enabled: false
19   # DEMO DB
18   # https://neon.com/
17   # Neon development database.
16   # Note: Outside of a project,
15   # sensitive information (username & password)
14   # should be stored securely.
13   datasource:
12     url: jdbc:postgresql://ep-noisy-thunder-alhe8xvg.0.us-east-1.aws.neon.tech/neondb?sslmode=require
11     username: neondb_owner
10     password: #####PLACEHOLDER_PUT_PASSWORD_HERE#####
9   # Development JPA settings.
8   jpa:
7     hibernate:
6       ddl-auto: update
5     show-sql: true
4   # Template settings.
3   thymeleaf:
2     cache: false
1   # Static resource caching.
26 web: You, 2 weeks ago • kommentare yaml ...
1   resources:
2     cache:
3       period: 0
4       cachecontrol:
5         no-cache: true
6         must-revalidate: true
7
```

Hinweis: Alle folgenden Bilder befinden sich auch in digitaler, gut lesbarer Form im Projekt (Github) selbst.

Inhaltsverzeichnis

1	Einleitung	1
2	Ausgangssituation, Zielsetzung und Vorgehen	2
2.1	Problemstellung und Abgrenzung	2
2.2	Vorgehensmodell und Kausalitätsprinzip	3
3	Anforderungsanalyse	4
3.1	Stakeholder, Systemgrenze und Rollen	4
3.2	Anforderungsaufnahme	5
3.3	Funktionale Anforderungen	6
3.4	Nichtfunktionale Anforderungen	7
3.5	Anwendungsfälle	9
4	Systementwurf	10
4.1	Entwurf der Benutzungsoberfläche	10
4.2	Architekturentwurf	11
4.3	Datenmodell	12
4.4	Schnittstellenkonzept	14
5	Implementierung der Kernfunktionen	15
5.1	Authentifizierung und Sicherheit	15
5.2	Marktdatenabruf und Fallback-Verhalten	16
5.3	Kauf- und Verkaufslogik	18
5.4	Portfolio- und Performanceberechnung	19
5.5	Benchmark-, EZB- und Inflationslogik	21
5.6	Benachrichtigungen, Aktivitäten und Einstellungen	22
6	Testkonzept und Validierung	24
7	Technische Inbetriebnahme und Nutzungseinführung	27
7.1	Technische Inbetriebnahme	27
7.2	Schulungsunterlage: Migration von Neon zu einer eigenen PostgreSQL-Datenbank .	28
7.3	Einführung der Nutzung	30
8	Fazit und Ausblick	31

1 Einleitung

Diese Ausarbeitung beschreibt die Entwicklung einer webbasierten Anwendung zur privaten Verwaltung eines Aktien- und Krypto-Portfolios. Das Produkt trägt den Arbeitstitel „Stock Tracker“ beziehungsweise „Aktien-Tracking“ und verfolgt das Ziel, Kauf- und Verkaufsvorgänge, aktuelle Bestandswerte, historische Transaktionen und einfache Auswertungen in einer gemeinsamen Anwendung bereitzustellen. Die Anwendung richtet sich an private Nutzerinnen und Nutzer, die eigene Wertpapier- und Kryptopositionen nachvollziehbar dokumentieren möchten, ohne dafür eine komplexe professionelle Handelsplattform verwenden zu müssen. Im Mittelpunkt steht deshalb nicht die Ausführung echter Börsenorders, sondern die strukturierte Dokumentation bereits getätigter oder manuell erfasster Portfolioentscheidungen.

Die Problemstellung ergibt sich aus mehreren Beobachtungen. Viele private Anleger erfassen ihre Transaktionen in Tabellenkalkulationen, verlieren dabei aber schnell den Zusammenhang zwischen Kaufpreis, aktueller Marktbewertung, Bestandsmenge, realisierten Verkäufen und historischer Entwicklung. Gleichzeitig liefern Depotanbieter zwar Kursdaten, aber nicht immer die persönlich gewünschte Sicht auf Performance, Export, Aktivitätshistorie oder Vergleichswerte. Die Anwendung soll diese Lücke schließen, indem sie eine übersichtliche Weboberfläche mit einer persistenten Datenhaltung und externen Marktdaten kombiniert.

Technisch wurde die Anwendung als Spring-Boot-Webanwendung mit Java, Spring MVC, Spring Security, Spring Data JPA, Thymeleaf, PostgreSQL und modular aufgebautem JavaScript realisiert. Diese Technologiewahl passt zur Aufgabenstellung, weil serverseitig gerenderte HTML-Seiten eine robuste und überschaubare Architektur erlauben, während REST-nahe JSON-Endpunkte die dynamischen Bestandteile des Frontends versorgen. Die Daten werden nutzerbezogen gespeichert, wodurch mehrere Anwender getrennte Portfolios verwalten können.

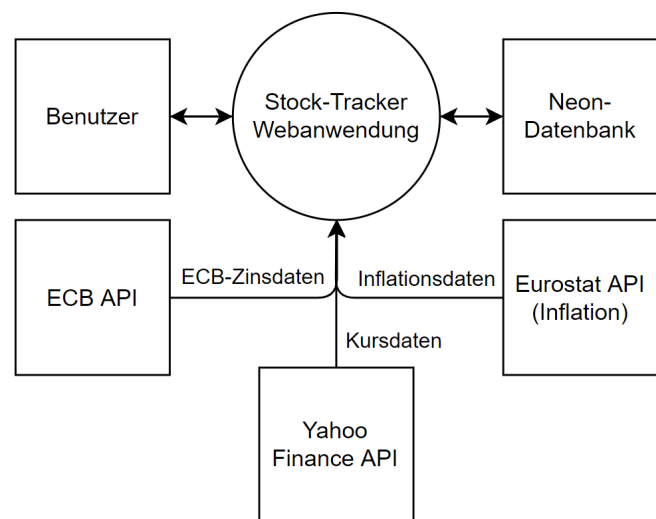


Abbildung 1.1: Visualisierung der verallgemeinerten Anwendung

Aus dem Ziel der nachvollziehbaren Portfolioverwaltung folgen die weiteren Arbeitsschritte der Ausarbeitung. Zunächst werden Ausgangssituation, Stakeholder und Anforderungen beschrieben. Daraus werden Anwendungsfälle abgeleitet, die wiederum die Oberfläche, das Datenmodell, die Architektur und die zentralen Algorithmen begründen. Anschließend werden Tests, Inbetriebnahme und Nutzungseinführung dargestellt. Damit folgt die Arbeit dem Kausalitätsprinzip: Jede technische Entscheidung wird aus einem vorherigen fachlichen Bedarf hergeleitet.

2 Ausgangssituation, Zielsetzung und Vorgehen

2.1 Problemstellung und Abgrenzung

Die Ausgangssituation der Anwendung ist ein typisches Szenario privater Portfolioverwaltung. Eine Nutzerin oder ein Nutzer besitzt verschiedene Wertpapiere, beispielsweise Aktien, ETFs oder Kryptowährungen. Diese Positionen wurden zu unterschiedlichen Zeitpunkten, mit unterschiedlichen Mengen und zu unterschiedlichen Preisen gekauft. Später können weitere Käufe hinzukommen oder Teilverkäufe erfolgen. Für eine vernünftige Bewertung reicht es deshalb nicht aus, nur den aktuellen Bestand zu kennen. Wichtig ist auch, wie dieser Bestand entstanden ist, welche Transaktionen ihn beeinflusst haben und welche Performance sich daraus gegenüber dem investierten Kapital ergibt. Eine manuelle Lösung in einer Tabellenkalkulation ist grundsätzlich möglich, führt aber zu mehreren Nachteilen. Kursdaten müssen regelmäßig manuell aktualisiert oder über externe Funktionen eingebunden werden. Die Berechnung durchschnittlicher Einstandskurse, aktueller Werte und prozentualer Renditen muss selbst gepflegt werden. Fehler in Formeln oder versehentlich überschriebene Werte können die Aussagekraft der Tabelle verschlechtern. Auch eine nutzerbezogene Zugriffskontrolle, ein Aktivitätsprotokoll, ein Export oder eine moderne Weboberfläche entstehen nicht automatisch. Genau hier setzt der Stock Tracker an.

Die fachliche Problemstellung lässt sich als Bedarf an einer konsistenten, datenbankgestützten Portfolioansicht formulieren. Benutzer sollen sich registrieren und anmelden können. Danach sollen sie Assets suchen, Kaufvorgänge erfassen, vorhandene Positionen verkaufen, offene Bestände anzeigen, Transaktionen filtern und relevante Auswertungen abrufen können. Die Anwendung soll aktuelle Kurse einbeziehen, damit Bestandswerte nicht ausschließlich auf historischen Kaufdaten beruhen. Gleichzeitig soll das System erkennen, wenn sich ein angezeigter Preis zwischen Frontend-Vorschau und Backend-Verarbeitung deutlich verändert hat.

Aus dieser Problemstellung folgt eine klare Abgrenzung: Die Anwendung ist kein Broker, sondern ein Dokumentations- und Analysewerkzeug. Regulatorische und sicherheitskritische Anforderungen reduzieren sich dadurch, während Datenkonsistenz, Nachvollziehbarkeit, Benutzertrennung und korrekte Berechnungen wesentlich bleiben. Transaktionen werden folglich in der eigenen PostgreSQL-Datenbank (in der Entwicklung über Neon) gespeichert, ohne externen Handelsauftrag.

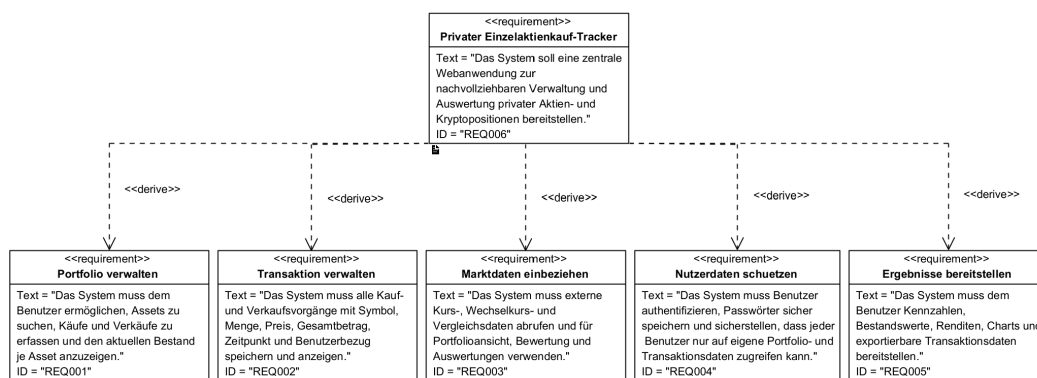


Abbildung 2.1: Übersicht der Hauptanforderungen

2.2 Vorgehensmodell und Kausalitätsprinzip

Für die Bearbeitung der Projektarbeit wurde ein inkrementelles Vorgehen gewählt. Eine streng lineare Wasserfallumsetzung wäre für eine Webanwendung mit Oberfläche, Datenbank, externer API-Anbindung und Berechnungslogik zu starr gewesen. Ein rein agiles Vorgehen ohne dokumentierte Zwischenergebnisse wäre dagegen für die Ausarbeitung ungeeignet, weil Anforderungen, Entwurf, Implementierung und Tests nachvollziehbar begründet werden müssen. Deshalb wurden Anforderungen, Anwendungsfälle, Oberfläche, Datenmodell, Services und Tests schrittweise konkretisiert und bei Bedarf rückgekoppelt.

Das Kausalitätsprinzip wurde während der Entwicklung des Projektes in den Vordergrund gestellt. Aus der Problemstellung entsteht der Bedarf nach nutzerbezogener Portfolioverwaltung. Daraus folgen Anforderungen wie Registrierung, Login, Asset-Suche, Kauf, Verkauf, Portfolioansicht, Transaktionshistorie und Export. Diese Anforderungen begründen die Anwendungsfälle; die Anwendungsfälle begründen Oberfläche und Datenmodell; aus Datenmodell und fachlichen Regeln entstehen zentrale Algorithmen wie die Bestandsfortschreibung und die Portfolio-Performanceberechnung.

Technische Entscheidungen werden dadurch nicht isoliert getroffen. PostgreSQL wird eingesetzt, weil persistente, relationale und transaktionale Datenhaltung benötigt wird. Spring Security folgt aus dem Schutz persönlicher Portfoliodaten. Thymeleaf und JavaScript wurden gewählt, weil sie für den Projektumfang eine ausreichend dynamische, aber noch überschaubare Webanwendung ermöglichen.

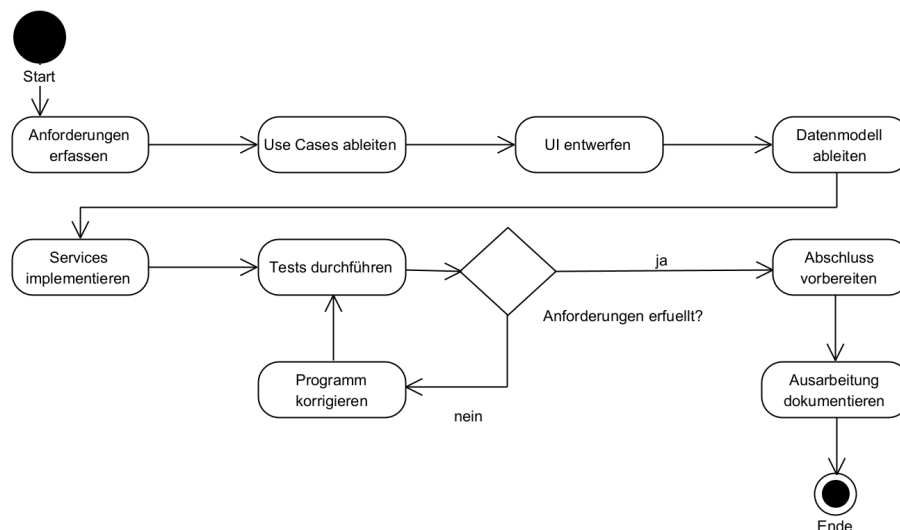


Abbildung 2.2: Planung des Projektablaufs

Alternativen wurden geprüft: Eine SPA (Single-Page-Application) mit Angular/React hätte Frontend und Backend stärker getrennt, aber den Integrationsaufwand erhöht, eine Desktopanwendung, die Webzugänglichkeit eingeschränkt; eine lokale Dateiablage, die relationale Konsistenz geschwächt. Die gewählte Architektur balanciert somit Umfang, Wartbarkeit, Funktionalität und technische Tiefe.

3 Anforderungsanalyse

3.1 Stakeholder, Systemgrenze und Rollen

Die wichtigste Stakeholdergruppe der Anwendung sind private Anwenderinnen und Anwender, die ihr eigenes Portfolio verwalten möchten. Sie erwarten eine verständliche Bedienung, eine schnelle Übersicht über Bestandswerte, eine nachvollziehbare Transaktionshistorie und eine verlässliche Trennung ihrer Daten von anderen Benutzern. Daneben gibt es aus Projektsicht die Entwickler, die eine wartbare technische Struktur benötigen.

Die Systemgrenze umfasst die Webanwendung selbst, ihre PostgreSQL-Datenbankanbindung und die von ihr angesprochenen externen Datenquellen. Innerhalb des Systems liegen Authentifizierung, Benutzerverwaltung, Portfolioverwaltung, Transaktionsverarbeitung, Aktivitätsprotokoll, Benachrichtigungen, Chartdatenaufbereitung sowie serverseitige Seitenlogik und clientseitige UI-Logik. Außerhalb des Systems liegen reale Börsen, Broker, Zahlungsströme und die externe Bereitstellung von Kurs-, Währungs-, EZB- und Inflationsdaten. Die Neon-Datenbank wird als externer PostgreSQL-Dienst angebunden, gehört fachlich aber zur persistenten Datenhaltung der Anwendung. Diese Abgrenzung ist entscheidend, weil die Anwendung keine rechtlich verbindlichen Handelsaktionen auslöst. Sie verarbeitet nur Benutzereingaben und externe Informationsdaten, keine tatsächlichen AN-/Verkäufe.

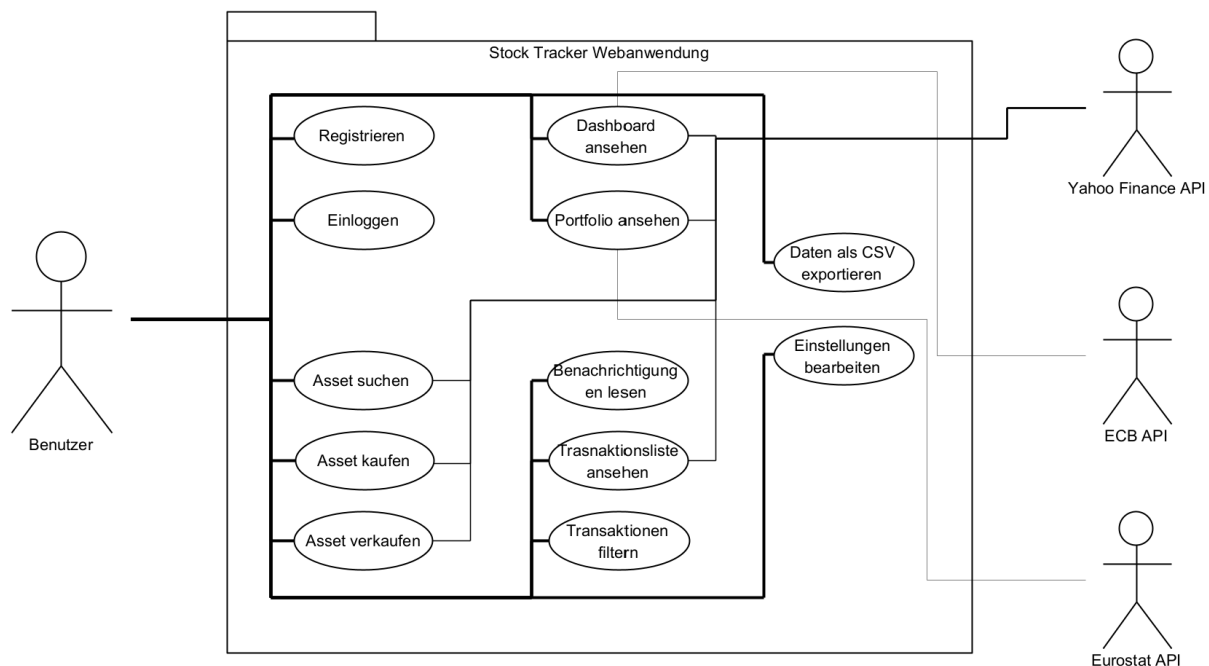


Abbildung 3.1: Wesentliche Funktionen für den Benutzer + Einflussort der externen APIs

Aus den Stakeholdern ergeben sich die wichtigsten Qualitätsanforderungen: Benutzer benötigen Datenschutz, Verfügbarkeit und eine nachvollziehbare Historie; Entwickler benötigen eine klare Schich-

tung, damit Controller, Services, Repositories, Entities und DTOs testbar bleiben. Die zentrale Rolle innerhalb der Anwendung ist der authentifizierte Benutzer. Nicht angemeldete Personen dürfen nur öffentliche Seiten wie Login und Registrierung erreichen. Eine Administratorrolle ist im aktuellen Stand bewusst nicht vorgesehen, weil die Anwendung auf private Portfolioverwaltung und nicht auf Benutzeradministration ausgerichtet ist.

3.2 Anforderungsaufnahme

Die Anforderungen wurden aus der beschriebenen Problemstellung, aus typischen Nutzungssituationen privater Portfolioverwaltung und aus der vorhandenen technischen Umsetzung abgeleitet. Die Anforderungsaufnahme konzentriert sich auf die Frage, welche Informationen ein Benutzer erfassen, anzeigen und auswerten möchte. Daraus ergeben sich mehrere Kernbereiche: Benutzerkonto, Portfolio, Transaktionen, Marktdaten, Auswertung, Benachrichtigungen, Einstellungen und technischer Betrieb. Diese Bereiche bilden die Grundlage für die spätere Struktur der Anwendung.

Ein erster fachlicher Bedarf ist die sichere Identifikation eines Benutzers. Da jedes Portfolio persönliche Daten enthält, muss die Anwendung Benutzerkonten unterstützen. Daraus folgen Registrierung, Login, Logout, Passwortspeicherung als Hash und Zugriffsbeschränkung für geschützte Seiten. Ein zweiter Bedarf ist die Erfassung von Assets. Der Benutzer soll nicht alle Stammdaten manuell eingeben müssen, sondern Assets über ein Symbol suchen und anhand externer Kursdaten bewerten lassen. Daraus folgt die Integration einer Marktdatenschnittstelle.

Ein dritter Bedarf betrifft die Transaktionsverarbeitung. Käufe müssen Menge, Symbol, Assetname, Preis, Gesamtbetrag, Zeitpunkt und Status speichern. Verkäufe müssen prüfen, ob ein ausreichender Bestand vorhanden ist. Der Bestand selbst darf nicht frei geraten werden, sondern muss aus Käufen und Verkäufen konsistent fortgeschrieben werden. Ein vierter Bedarf betrifft Auswertungen. Der Benutzer soll offene Positionen mit aktuellem Wert, Einstandswert und Performance sehen. Außerdem soll ein Chart die zeitliche Entwicklung und Vergleichswerte darstellen.

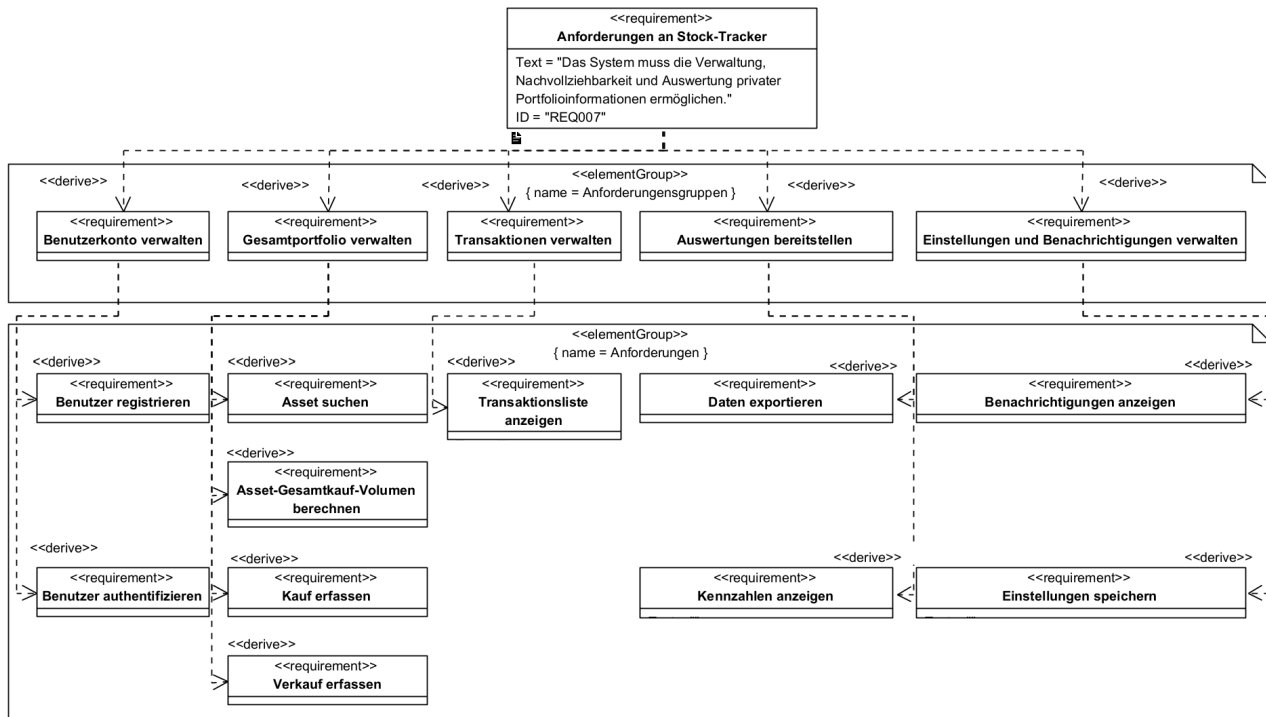


Abbildung 3.2: Anforderungsstruktur im Gesamtkontext

Die Anforderungsaufnahme führt auch zu nichtfunktionalen Anforderungen. Die Anwendung muss Daten persistent speichern, Berechnungen mit geeigneter Genauigkeit durchführen, Fehler externer Schnittstellen abfangen und eine bedienbare Oberfläche bereitstellen. Besonders wichtig ist die Trennung nach Benutzer, damit ein angemeldeter Benutzer ausschließlich seine eigenen Daten sieht. Weiterhin muss die Anwendung testbar sein. Deshalb sind Backend-Tests für Services, Controller, Repositories, Entities, DTOs, Exceptions und Security sowie JavaScript-Tests für UI-Hilfsfunktionen und Tabellenlogik vorgesehen. Aus diesen Anforderungen folgt die technische Schichtung, die im Architekturkapitel beschrieben wird.

3.3 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben, welche konkreten Leistungen das System bereitstellen muss. Eine erste Anforderung ist die Registrierung neuer Benutzer. Dabei müssen E-Mail-Adresse, Passwort, Passwortbestätigung sowie optionale Namensdaten verarbeitet werden. Die E-Mail-Adresse muss eindeutig sein, damit sie als Benutzername für den Login dienen kann. Das Passwort darf nicht im Klartext gespeichert werden, sondern wird über einen Passwortencoder gehasht. Nach der Registrierung wird ein Sicherheitscode erzeugt, der für die Passwort-zurücksetzen-Funktion verwendet werden kann.

Eine zweite Anforderung ist die Authentifizierung. Benutzer müssen sich mit E-Mail und Passwort anmelden können. Bei erfolgreichem Login werden sie zum Dashboard weitergeleitet. Bei einem

Fehler erscheint eine passende Meldung, ohne sicherheitskritisch offenzulegen, ob die E-Mail oder das Passwort falsch war. Nach dem Logout wird die Sitzung beendet. Alle fachlichen Seiten und API-Endpunkte für Portfolio, Transaktionen, Benachrichtigungen, Aktivitäten und Einstellungen müssen eine Anmeldung voraussetzen.

Eine dritte Anforderung betrifft die Portfolioverwaltung. Benutzer müssen Assets suchen, Kursdaten abrufen, Käufe erfassen und Bestände anzeigen können. Beim Kauf wird entweder der aktuelle Kurs oder ein benutzerdefinierter Ausführungspreis verwendet. Das System muss den Gesamtbetrag berechnen und den offenen Bestand erzeugen oder aktualisieren. Beim Verkauf muss das System prüfen, ob die verkaufte Menge den Bestand nicht übersteigt. Ist nach einem Verkauf die Menge null, wird der offene Bestand entfernt.

Eine vierte Anforderung ist die Darstellung von Auswertungen. Das Dashboard zeigt aktuelle Gesamtwerte, Rendite, Chartdaten, Allokation und letzte Aktivitäten. Die Portfolioside ergänzt diese Sicht um offene Positionen, wöchentliche Veränderungen sowie beste und schlechteste Positionen. Die Transaktionsseite soll alle Buchungen historisch darstellen und filterbar machen. Die Anwendung muss Aktivitäten protokollieren, zum Beispiel Trades oder Exportvorgänge. Benachrichtigungen sollen ungelesen gezählt, gelesen markiert und gelöscht werden können. Diese Anforderungen begründen die Serviceklassen `PortfolioService`, `TransactionService`, `ActivityService` und `NotificationService`.

Ergänzend muss die Anwendung Benutzereingaben validieren und fachliche Fehler verständlich zurückmelden. Dazu gehören doppelte Registrierung, ungültige Mengen, fehlende Holdings, zu grosse Preisabweichungen und nicht verfügbare externe Kursdaten.

3.4 Nichtfunktionale Anforderungen

Neben den sichtbaren Funktionen sind nichtfunktionale Anforderungen wesentlich für die Qualität des Produkts. Die erste zentrale Anforderung ist Sicherheit. Da Benutzerkonten und Finanzinformationen verarbeitet werden, müssen Zugriffsschutz, Passwort-Hashing und CSRF-Schutz realisiert werden. Spring Security schützt alle nicht öffentlichen Routen. Passwörter werden mit BCrypt gespeichert. Formular- und zustandsverändernde API-Aktionen werden durch CSRF-Mechanismen abgesichert. Dadurch wird verhindert, dass unberechtigte Personen auf Portfolioinformationen zugreifen oder fremde Aktionen auslösen.

Die zweite nichtfunktionale Anforderung ist Datenkonsistenz. Käufe, Verkäufe, Bestandsänderungen und Aktivitätsprotokolle müssen in einer fachlich konsistenten Reihenfolge verarbeitet werden. Deshalb liegen Kauf- und Verkaufsvorgänge in einem transaktionalen Service. Wird ein Kauf gespeichert, muss auch der zugehörige Bestand aktualisiert werden. Wird ein Verkauf gespeichert, muss der Bestand reduziert oder entfernt werden. Fehler sollen nicht zu halbfertigen Zuständen führen. Diese Anforderung begründet den Einsatz relationaler Datenhaltung und transaktionaler Service-Methoden.

Eine dritte Anforderung ist Genauigkeit bei Geld- und Mengenwerten. Die Anwendung verwendet `BigDecimal`, weil Gleitkommazahlen für Finanzwerte ungeeignet sind. Mengen und Preise werden mit definierten Skalen gespeichert, Gesamtbeträge auf zwei Nachkommastellen gerundet. Performancewerte werden ebenfalls bewusst gerundet, damit die Darstellung nachvollziehbar bleibt.

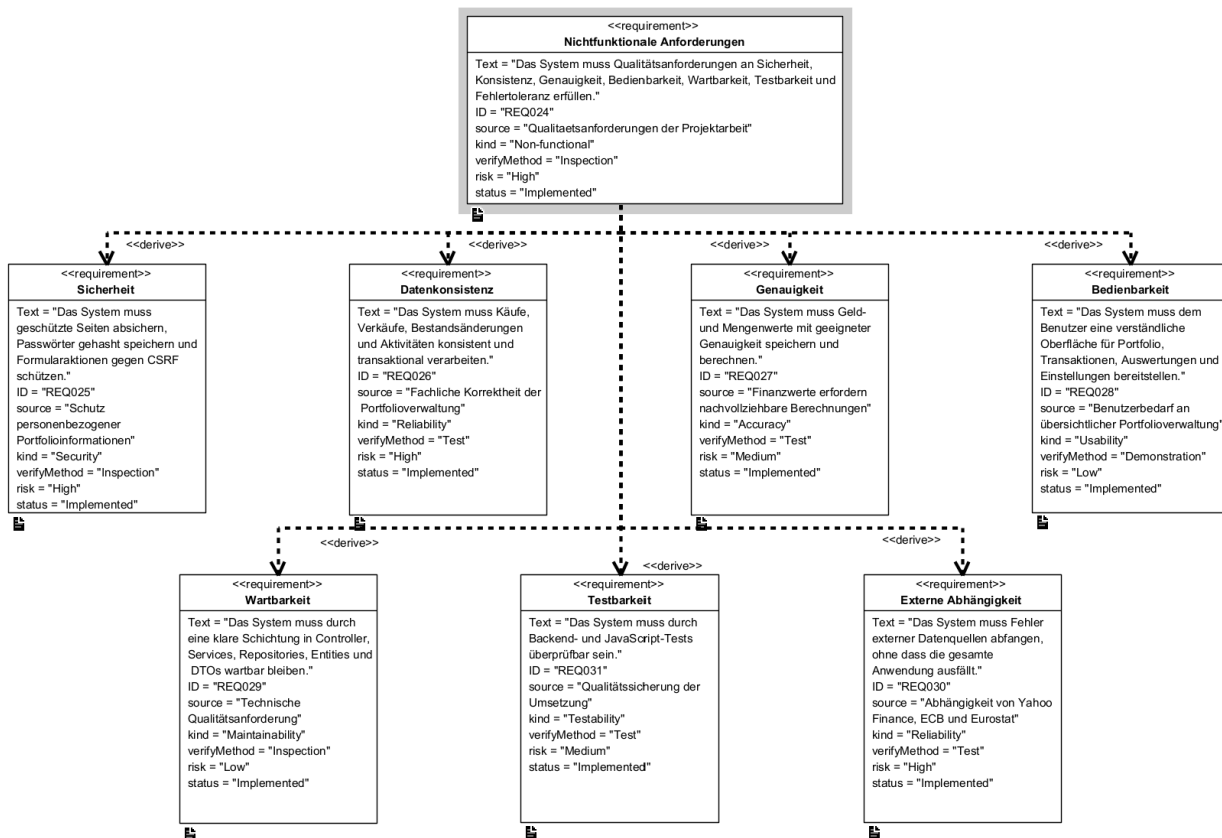


Abbildung 3.3: Die nichtfunktionalen Anforderungen an die Anwendung

Weitere Anforderungen betreffen Bedienbarkeit und Wartbarkeit. Die Anwendung soll ohne Frontend-Bundler laufen und trotzdem dynamische UI-Elemente bieten. Deshalb wird Thymeleaf für Seitenstruktur und JavaScript für interaktive Tabellen, Charts, Modale und Exporte verwendet. Wartbarkeit entsteht durch Schichtung: Controller nehmen Anfragen entgegen, Services enthalten Fachlogik, Repositories kapseln Datenbankzugriff, Entities bilden Datenstrukturen ab, DTOs begrenzen die Kommunikation mit dem Frontend. Die letzte wichtige Anforderung ist Fehlertoleranz gegenüber externen Schnittstellen. Wenn Yahoo Finance oder andere externe Dienste nicht erreichbar sind, soll die Anwendung nicht vollständig ausfallen, sondern leere Listen, Nullwerte oder zwischengespeicherte Daten verwenden.

Auch die Erweiterbarkeit ist relevant. Neue Auswertungen, weitere Datenquellen oder zusätzliche Benachrichtigungstypen sollen möglich sein, ohne die Kernlogik für Benutzer, Holdings und Transaktionen neu entwerfen zu müssen.

3.5 Anwendungsfälle

Aus den Anforderungen ergeben sich konkrete Anwendungsfälle. Der erste Anwendungsfall ist die Registrierung. Ein nicht angemeldeter Benutzer öffnet die Registrierungsseite, gibt E-Mail, Passwort, Passwortbestätigung und optionale Namensdaten ein und sendet das Formular ab. Das System validiert die Eingaben, prüft die Eindeutigkeit der E-Mail-Adresse, speichert den Benutzer mit gehashtem Passwort und erzeugt Standardeinstellungen. Danach erhält der Benutzer seinen Sicherheitscode und kann zum Login wechseln. Dieser Anwendungsfall begründet die Entitäten User und UserSettings sowie den UserService.

Der zweite Anwendungsfall ist die Anmeldung. Der Benutzer gibt E-Mail und Passwort ein. Spring Security lädt den Benutzer über den UserService, vergleicht das Passwort mit dem gespeicherten Hash und erzeugt bei Erfolg eine Sitzung. Danach kann der Benutzer geschützte Seiten aufrufen. Dieser Ablauf begründet die Sicherheitskonfiguration und die Trennung öffentlicher und geschützter Routen.

Der dritte Anwendungsfall ist der Kauf eines Assets. Der Benutzer sucht ein Asset, betrachtet den aktuellen Kurs, gibt Menge und gegebenenfalls ein Kaufdatum oder einen benutzerdefinierten Preis ein und bestätigt den Kauf. Das System holt aktuelle Kursdaten, prüft bei Livepreisen die Preisabweichung zur UI-Vorschau, berechnet den Gesamtbetrag, speichert die Transaktion und aktualisiert den Bestand. Dieser Anwendungsfall ist besonders wichtig, weil aus ihm die zentrale Transaktionslogik hervorgeht.

Der vierte Anwendungsfall ist der Verkauf. Der Benutzer wählt eine vorhandene Position und gibt eine Verkaufsmenge ein. Das System prüft, ob genügend Bestand vorhanden ist, ruft den aktuellen Kurs ab, berechnet den Gesamtbetrag, speichert die Verkaufstransaktion und reduziert den Bestand. Wird die gesamte Menge verkauft, wird die offene Position gelöscht. Weitere Anwendungsfälle sind das Anzeigen von Dashboardkennzahlen, das Filtern der Transaktionen, das Exportieren von Daten, das Lesen von Benachrichtigungen, das Ändern von Einstellungen und das Anfordern einer Kontolöschung.

4 Systementwurf

Die Anforderungsanalyse legt fest, welche fachlichen Leistungen die Anwendung bereitstellen muss. Daraus folgt der Systementwurf: Er beschreibt, wie Oberfläche, Architektur und Datenmodell so zusammenspielen, dass die Anforderungen technisch umsetzbar und prüfbar werden.

4.1 Entwurf der Benutzungsoberfläche

Die Benutzungsoberfläche wurde als klassische Web-Oberfläche mit mehreren fachlichen Seiten entworfen. Diese Entscheidung folgt aus den identifizierten Anwendungsfällen. Da Benutzer wiederholt zwischen Übersicht, Portfolio, Transaktionen und Einstellungen wechseln, ist eine feste Navigationsstruktur sinnvoll. Die Anwendung verwendet eine Sidebar und eine Topnavigation für geschützte Seiten. Login und Registrierung besitzen dagegen eigene, reduzierte Layouts, weil dort noch keine Portfoliofunktionen verfügbar sind.

Das Dashboard dient als Einstieg nach dem Login. Es soll dem Benutzer schnell zeigen, wie sich sein Portfolio entwickelt und welche Aktivitäten zuletzt stattgefunden haben. Die Portfolioseite konzentriert sich auf offene Positionen, aktuelle Werte, Performance, Allokation und Verkaufsaktionen. Die Transaktionsseite bildet die historische Perspektive ab. Sie zeigt Käufe und Verkäufe tabellarisch, unterstützt Filter nach Typ, Suche, Datum und CSV-Export. Die Einstellungsseite erlaubt Profil- und Anzeigeoptionen wie Dark Mode, Preisalarme und Transaktionsupdates. Die Automationsseite ist als Bereich für erweiterte Funktionen vorgesehen.

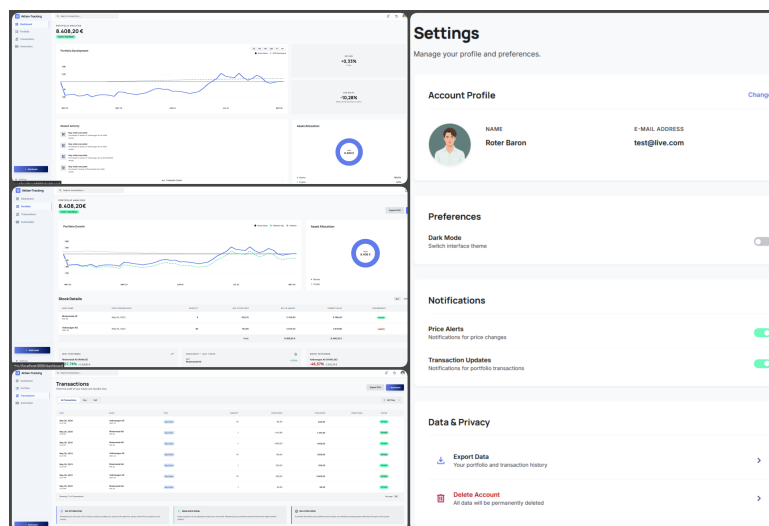


Abbildung 4.1: UI der Webanwendung (Stand: Release v1.0)

Die Oberfläche nutzt wiederverwendbare Thymeleaf-Fragmente für Kopfbereich, Sidebar, Topnavigation, Skripte und Asset-Hinzufügen-Dialog. Diese Fragmentierung reduziert Dopplungen und sorgt für ein einheitliches Layout. Die dynamischen Interaktionen werden durch modular organisierte JavaScript-Dateien umgesetzt. Beispielsweise besitzen Portfolio, Dashboard, Transaktionen,

Einstellungen und Add-Asset-Dialog jeweils eigene Skriptmodule. Diese Struktur passt zur fachlichen Seitentrennung und verhindert, dass eine einzige grosse JavaScript-Datei alle Aufgaben übernimmt. Ein wichtiger Entwurfsaspekt ist die Rückmeldung an den Benutzer. Beim Kauf oder Verkauf werden Preise, Mengen und Gesamtwerte angezeigt. Wenn sich ein Livepreis zu stark verändert, wird die Aktion nicht stillschweigend mit einem unerwarteten Preis abgeschlossen, sondern durch eine `PriceChangedException` abgesichert. Tabellen zeigen leere Zustände, wenn keine Daten vorhanden sind. Filter und Pagination verbessern die Bedienbarkeit bei größeren Transaktionslisten. Damit folgt die Oberfläche nicht nur optischen, sondern fachlichen Anforderungen: Sie macht Portfolioentscheidungen nachvollziehbar und reduziert Fehlbedienung.

4.2 Architekturentwurf

Die Anwendung ist in einer mehrschichtigen Spring-Boot-Architektur umgesetzt. Diese Architektur ergibt sich aus dem Bedarf, Webanfragen, fachliche Logik, Datenzugriff und Darstellung sauber zu trennen. Die Controller-Schicht verarbeitet HTTP-Anfragen und ruft passende Services auf. Die Service-Schicht enthält die fachlichen Regeln, etwa Kauf, Verkauf, Berechnung von Bestandswerten, Kursdatenabruf, Benachrichtigungen und Benutzerverwaltung. Die Repository-Schicht nutzt Spring Data JPA für den Zugriff auf relationale Daten. Die Entity-Schicht bildet die persistenten Datenobjekte ab. DTOs begrenzen, welche Daten zwischen Backend und Frontend ausgetauscht werden.

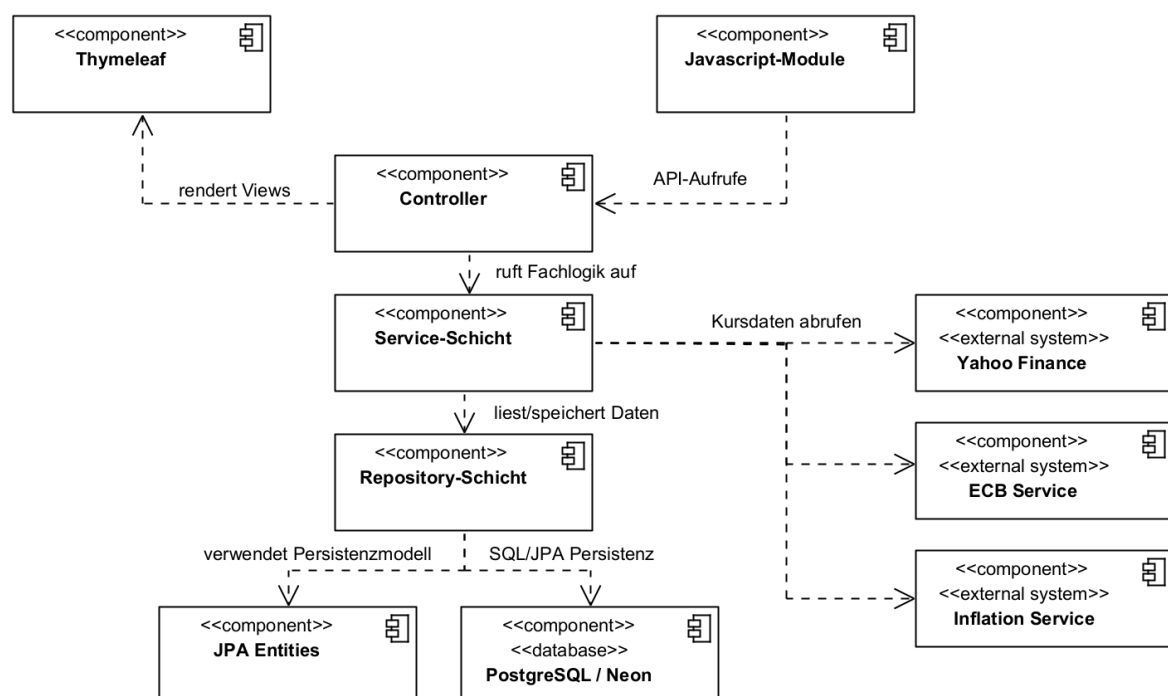


Abbildung 4.2: Systemarchitektur/Komponenten des Stock Trackers

Diese Schichtung wurde gewählt, weil sie Testbarkeit und Wartbarkeit verbessert. Controller können mit MockMVC oder ähnlichen Testwerkzeugen geprüft werden, ohne echte externe Dienste zu benötigen. Services können mit gemockten Repositories und Marktdatendiensten getestet werden. Repositories können isoliert mit H2-In-Memory-Datenbank getestet werden. JavaScript-Module können mit Jest und jsdom geprüft werden. Dadurch wird die Anwendung nicht nur implementiert, sondern auch strukturiert validierbar.

Die Backend-Controller spiegeln die fachlichen Bereiche. AuthController behandelt Registrierung, Login-nahe Seiten, Profil, Einstellungen und Kontolöschung. PageController liefert die geschützten HTML-Seiten. AssetController stellt API-Endpunkte für Asset-Suche, Kurse, Käufe, Verkäufe, Holdings, Transaktionen und Chartdaten bereit. StockController bietet Kurs- und Historienendpunkte. NotificationController, ActivityController und InflationController kapseln weitere fachliche API-Bereiche. Diese Aufteilung folgt den Use Cases und verhindert, dass ein einzelner Controller zu viele Verantwortlichkeiten übernimmt.

Eine reine Single-Page-Application wurde nicht gewählt, weil sie für den Projektumfang einen zusätzlichen Frontend-Build, Routing im Browser und eine strengere API-Trennung erfordert hätte. Gleichzeitig wurde auch kein einzelner monolithischer Controller verwendet, weil dadurch Authentifizierung, Portfolioverwaltung, Marktdaten, Benachrichtigungen und Aktivitäten zu stark vermischt würden. Die gewählte Struktur trennt deshalb serverseitig gerenderte Seiten, REST-nahe JSON-Endpunkte und fachliche Services. Die SecurityConfig wirkt dabei quer zu den Controllern: Sie entscheidet nicht über einzelne Portfoliofunktionen, sondern legt übergreifend fest, welche Routen öffentlich sind und welche eine authentifizierte Sitzung benötigen.

Für den technischen Betrieb wird Maven als Build-Werkzeug genutzt. Java 17 bildet die Laufzeitbasis. PostgreSQL ist die Ziel-Datenbank für Entwicklung und Produktion, während H2 für schnelle Repositorytests eingesetzt wird. Thymeleaf rendert HTML-Templates serverseitig. Externe Marktdaten werden über HTTP von Yahoo Finance abgerufen. Diese Architektur ist nicht maximal verteilt, aber für den Projektumfang bewusst angemessen. Sie vermeidet die Komplexität separater Frontend- und Backend-Deployments, bietet aber trotzdem klare Schnittstellen und eine fachlich saubere Schichtung.

Damit bleibt die Anwendung überschaubar und dennoch realitätsnah. Sie nutzt etablierte Frameworks, trennt Verantwortlichkeiten und lässt sich sowohl lokal als auch mit einer Cloud-Datenbank betreiben.

4.3 Datenmodell

Das Datenmodell ergibt sich direkt aus den Anwendungsfällen. Da alle Portfolioinformationen nutzerbezogen sind, steht die Entität User im Zentrum. Ein Benutzer besitzt eine eindeutige E-Mail-Adresse, ein gehashtes Passwort, optionale Namensdaten, Aktivierungsstatus, Erstellungs- und Aktualisierungszeitpunkte sowie weitere Felder wie Sicherheitscode-Hash und Markierung zur Kontolöschung. Weil der Benutzer auch als Spring-Security-Principal verwendet wird, implementiert die Klasse UserDetails und stellt die Rolle ROLE_USER bereit.

Zur Speicherung offener Positionen dient die Entität Holding. Ein Holding gehört zu genau einem Benutzer und einem Asset-Symbol. Es speichert Symbol, Assetname, Menge, durchschnittliche

Einstandsbasis, erstes Kaufdatum und letzte Aktualisierung. Die eindeutige Kombination aus Benutzer und Symbol verhindert, dass für denselben Benutzer mehrere offene Bestandsdatensätze zum gleichen Asset entstehen. Diese Entscheidung folgt aus dem fachlichen Bedarf, pro Asset eine konsolidierte aktuelle Position anzuzeigen.

Transaktionen werden in der Entität *Transaction* gespeichert. Sie repräsentieren historische Ereignisse und enthalten Benutzer, Symbol, Assetname, Menge, Preis pro Anteil, Gesamtbetrag, Transaktionstyp, Status, Erstellungszeitpunkt und optional die ID eines zugehörigen Kaufs. Im Gegensatz zu Holdings bilden Transaktionen nicht nur den aktuellen Zustand ab, sondern die Historie. Diese Trennung ist wichtig: Ein Holding kann sich durch Verkauf reduzieren oder verschwinden, während die Transaktion als Nachweis erhalten bleibt.

Die wichtigste Modellentscheidung ist damit die Trennung zwischen Zustand und Ereignis. *Holding* beantwortet die Frage, welche Position ein Benutzer aktuell offen hält. *Transaction* beantwortet dagegen, durch welche Käufe und Verkäufe dieser Zustand entstanden ist. Beide Entitäten besitzen einen Benutzerbezug, damit Daten nicht nur über die Oberfläche, sondern bereits im Datenmodell mandantenähnlich getrennt werden. Die eindeutige Kombination aus Benutzer und Symbol bei Holdings verhindert doppelte offene Positionen, während mehrere Transaktionen pro Symbol ausdrücklich erlaubt sind, weil sie die Historie abbilden.

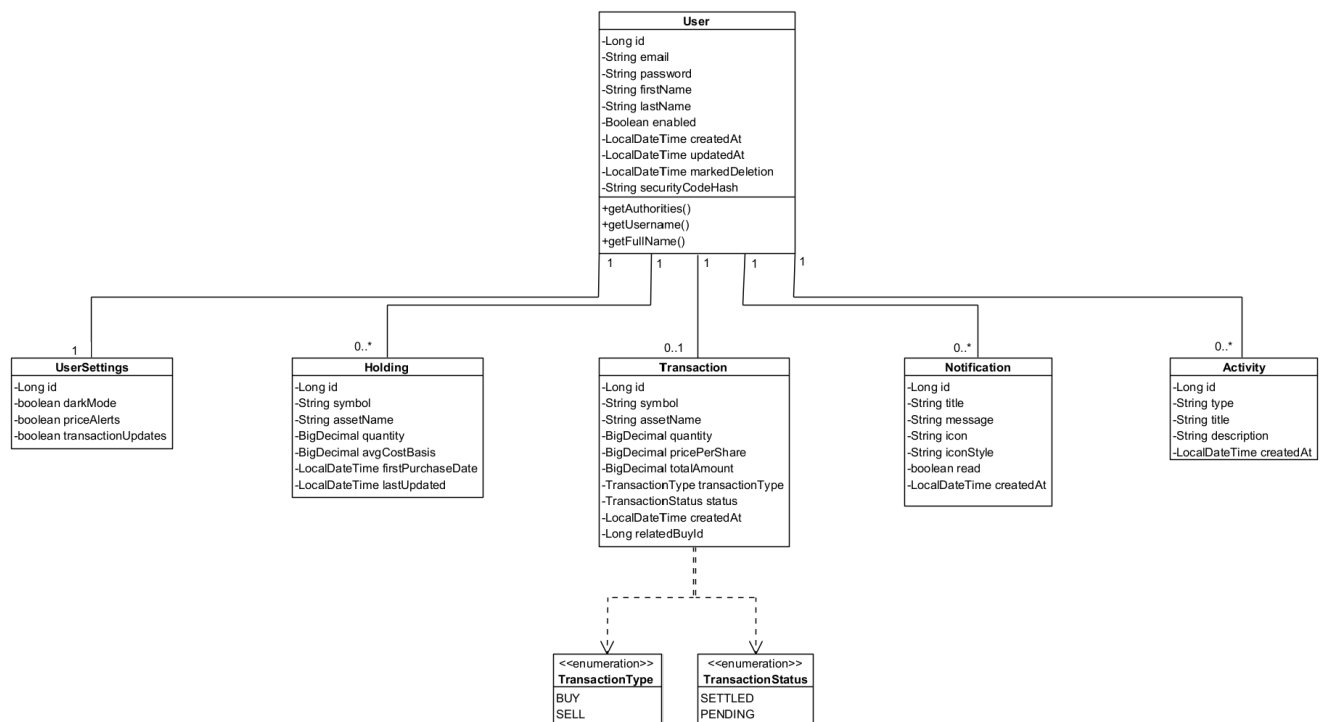


Abbildung 4.3: Entitäten des Stock Trackers

Weitere Entitäten ergänzen das Modell, ohne den Portfolio-Kern zu überladen. `UserSettings` speichert pro Benutzer Anzeige- und Benachrichtigungseinstellungen und ist deshalb eins zu eins mit `User` verbunden. `Notification` speichert sichtbare Meldungen mit Titel, Nachricht, Icon, Lese-status und Zeitpunkt. `Activity` speichert Bedien- und Systemereignisse wie Trades, Exporte oder Einstellungsänderungen. Diese Trennung verhindert, dass Transaktionen für allgemeine UI-Ereignisse zweckentfremdet werden. Die Enumerationen `TransactionType` und `TransactionStatus` begrenzen zulässige Werte und machen Kauf, Verkauf und abgeschlossene Buchungen explizit modellierbar. Insgesamt ist das Datenmodell normalisiert genug, um Redundanz zu begrenzen, aber pragmatisch genug, um typische Abfragen effizient zu bedienen.

4.4 Schnittstellenkonzept

Die Anwendung benötigt externe Marktdaten, damit Portfolioanzeigen nicht nur auf manuell gespeicherten Transaktionen beruhen. Deshalb sieht der Entwurf Schnittstellen zu Yahoo Finance sowie zu EZB- und Inflationsdatenquellen vor. Diese Systeme liegen außerhalb der eigenen Systemgrenze und werden im Projekt als Informationsquellen behandelt. Aus dieser Einordnung folgt, dass die Anwendung externe Ausfälle abfangen muss und gespeicherte Portfoliodaten weiterhin die verbindliche Grundlage bleiben.

5 Implementierung der Kernfunktionen

Nach dem Systementwurf beschreibt dieses Kapitel die Umsetzung der Funktionen, die für den fachlichen Nutzen der Anwendung entscheidend sind. Dazu gehören Zugriffsschutz, externe Datenbeschaffung, Transaktionsverarbeitung, Portfolioauswertung sowie begleitende Informationsfunktionen.

5.1 Authentifizierung und Sicherheit

Die Sicherheitsarchitektur ergibt sich aus der Anforderung, persönliche Portfoliodaten zu schützen. Der Zugriff auf fachliche Seiten und API-Endpunkte ist nur für angemeldete Benutzer erlaubt. Öffentlich erreichbar bleiben nur Startseite, Login, Registrierung, Passwort-zurücksetzen-Funktion, Favicon und statische Ressourcen. Diese Trennung wird in der SecurityConfig über eine SecurityFilterChain umgesetzt. Alles, was nicht ausdrücklich freigegeben ist, erfordert Authentifizierung. Dadurch entsteht ein Default-Deny-Verhalten, das für personenbezogene Anwendungen angemessen ist.

Die Anmeldung erfolgt formularbasiert. Benutzer geben E-Mail und Passwort ein. Spring Security verarbeitet den Login über den konfigurierten Login-Endpoint. Die E-Mail wird als Benutzername verwendet, das Passwort als Credential. Der UserService implementiert UserDetailsService und lädt den Benutzer anhand der E-Mail-Adresse aus der Datenbank. Das Passwort wird nicht direkt verglichen, sondern über einen BCrypt-PasswordEncoder geprüft. Die verwendete Stärke von 12 erhöht den Aufwand für Offline-Angriffe, bleibt aber für normale Logins noch praktikabel.

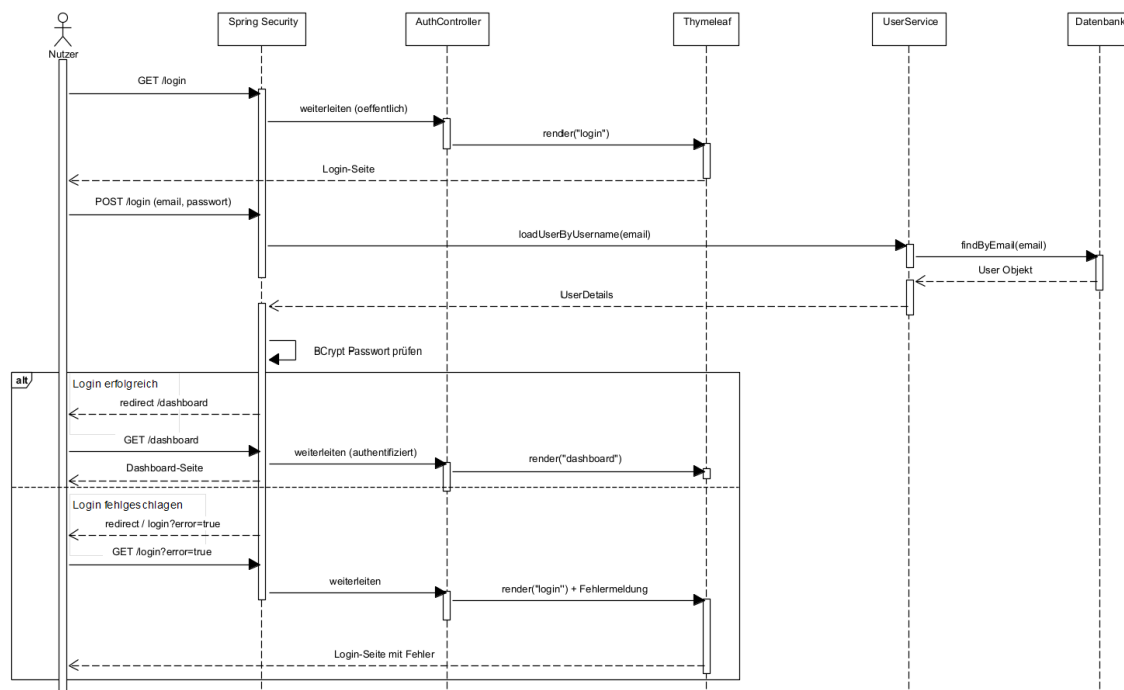


Abbildung 5.1: Ablauf des Login-Prozesses

CSRF-Schutz bleibt aktiviert. Das ist wichtig, weil die Anwendung formularbasierte und zustandsverändernde Aktionen enthält. Ohne CSRF-Schutz könnte ein Angreifer versuchen, eine angemeldete Sitzung des Benutzers für ungewollte Aktionen zu missbrauchen. Logout löscht die Session und das JSESSIONID-Cookie. Die Anwendung trennt dadurch Authentifizierung, Autorisierung und Sitzungspflege sauber von der fachlichen Portfolioverarbeitung.

Auch innerhalb der fachlichen Services wird Sicherheit berücksichtigt. API-Methoden erhalten den authentifizierten Benutzer als Principal. Repositories suchen Daten nutzerbezogen, etwa Holdings oder Transaktionen eines konkreten Benutzers. Dadurch soll verhindert werden, dass ein Benutzer Daten eines anderen Benutzers sieht. Die Kontolöschung wird nicht sofort physisch umgesetzt, sondern über eine Markierung vorbereitet. Dieses Vorgehen lässt sich später erweitern, beispielsweise um Bestätigungsfristen oder administrative Freigaben zu unterstützen. Insgesamt folgt die Sicherheitslösung dem Projektumfang: Sie bietet robuste Standardmechanismen, ohne eine überdimensionierte Rollen- oder Mandantenverwaltung einzuführen.

Wichtig ist außerdem, dass Sicherheitslogik nicht im Frontend verbaut wird. Die Oberfläche kann Buttons ausblenden, die verbindliche Zugriffskontrolle muss jedoch immer serverseitig in Spring Security und in den nutzerbezogenen Datenbankabfragen liegen.

5.2 Marktdatenabruf und Fallback-Verhalten

Die Anwendung benötigt aktuelle und historische Marktdaten, damit Portfolioanzeigen nicht nur auf manuell eingegebenen Werten beruhen. Daraus folgt die Anbindung externer Schnittstellen. Der StockService greift auf Yahoo-Finance-Endpunkte zu, um Kursdaten, historische Preisreihen, Asset-Suchergebnisse und Währungsumrechnungen zu erhalten. Für Chart- und Benchmarkfunktionen werden außerdem EZB- und Inflationsdaten über eigene Services eingebunden. Diese externen Datenquellen liegen außerhalb der Systemgrenze und müssen deshalb fehlertolerant behandelt werden.

Der Kursabruf erfolgt über HTTP-Anfragen mit definiertem User-Agent, Timeout und JSON-Verarbeitung. Für einen Einzelkurs werden Metadaten wie aktueller Marktpreis, vorheriger Schlusskurs, Volumen, Name und Währung ausgelesen. Für historische Daten werden Zeitstempel und OHLCV-Werte verarbeitet. Einträge ohne Schlusskurs werden übersprungen, damit Chartberechnungen nicht durch leere Werte verfälscht werden. Die Asset-Suche liefert bis zu acht Treffer mit Symbol, Name und Börsenplatz. Dadurch kann der Benutzer Assets komfortabler hinzufügen, ohne alle Symbole auswendig kennen zu müssen.

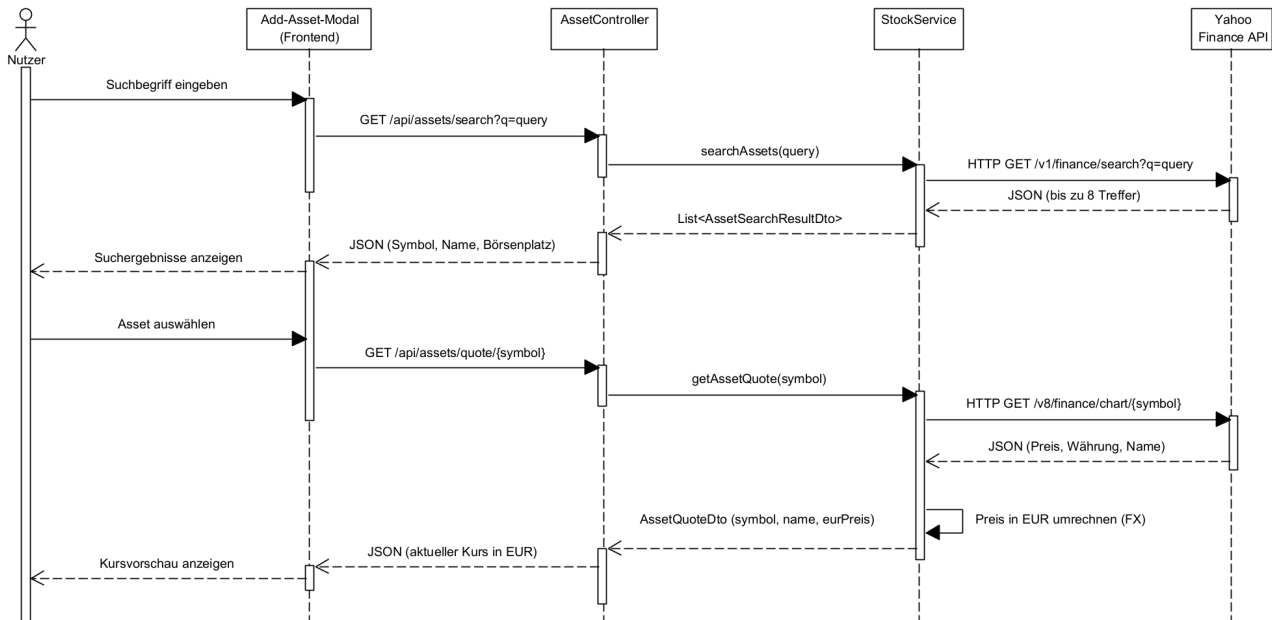


Abbildung 5.2: Ablauf der Asset-Suche

Ein wichtiger Entwurfsaspekt ist die Währungsumrechnung. Da Assets in unterschiedlichen Währungen notieren können, normalisiert `getAssetQuote` Preise auf Euro. Falls ein Asset nicht bereits in EUR notiert, wird ein FX-Kurs zur Umrechnung abgerufen. Diese FX-Kurse werden kurzzeitig gecacht, um unnötige externe Anfragen zu vermeiden. Wenn ein FX-Abruf scheitert, verwendet das System einen vorhandenen Cachewert. Das ist nicht perfekt, verhindert aber einen vollständigen Ausfall der Anwendung und erzeugt tendenziell nur kleine Unstimmigkeiten.

Die externen Schnittstellen begründen auch mehrere nichtfunktionale Entscheidungen. Die Anwendung darf nicht davon ausgehen, dass Yahoo Finance immer erreichbar ist oder vollständige Daten liefert. Deshalb geben Methoden bei Fehlern leere Listen, Nullwerte oder Fallback-Objekte zurück. Diese Entscheidung ist fachlich akzeptabel, weil der Stock Tracker keine echte Orderausführung vornimmt. Für eine professionelle Handelsanwendung wäre ein strengeres Fehlerverhalten erforderlich. Im Projektkontext ist die gewählte Fehlertoleranz angemessen, weil sie Bedienbarkeit und Stabilität verbessert.

Die Schnittstellen werden deshalb als Informationsquellen verstanden, nicht als autoritative Handelsinfrastruktur. Dieser Unterschied prägt Fehlerbehandlung, UI-Hinweise und die Abgrenzung des Produkts.

5.3 Kauf- und Verkaufslogik

Die Kauf- und Verkaufslogik ist der zentrale fachliche Kern der Anwendung. Sie wird im TransactionService umgesetzt. Beim Kauf ruft das System zunächst aktuelle Assetdaten ab. Danach wird der verwendete Preis bestimmt. Falls der Benutzer einen gültigen benutzerdefinierten Preis eingetragen hat, wird dieser verwendet. Andernfalls wird der aktuelle Kurs aus der Marktdaten-schnittstelle genutzt. Diese Unterscheidung ist wichtig, weil Benutzer historische oder abweichende Ausführungspreise dokumentieren können sollen.

Bei Livepreisen prüft das System, ob sich der aktuelle Preis zu stark von dem Preis unterscheidet, den die Oberfläche dem Benutzer zuvor angezeigt hat. Die Toleranz liegt bei einem Prozent. Wird diese Schwelle überschritten, wirft das System eine PriceChangedException. Damit wird verhindert, dass der Benutzer unbemerkt zu einem deutlich anderen Preis kauft oder verkauft, als er bei der Eingabe gesehen hat. Diese Regel folgt direkt aus der Anforderung an Nachvollziehbarkeit und Fehlbedienungsschutz.

Die Preisabweichungsprüfung ist besonders wichtig, weil zwischen Anzeige im Frontend und Verarbeitung im Backend Zeit vergeht. Ohne erneute Prüfung könnte die Oberfläche einen Preis suggerieren, der zum Zeitpunkt der Speicherung nicht mehr gilt. Die Anwendung entscheidet sich deshalb nicht für eine stille Korrektur, sondern für einen kontrollierten Abbruch mit Rückmeldung an den Benutzer. Diese Entscheidung passt zur fachlichen Abgrenzung der Anwendung: Da keine echten Börsenorders ausgeführt werden, ist Nachvollziehbarkeit wichtiger als eine automatische Ausführung um jeden Preis.

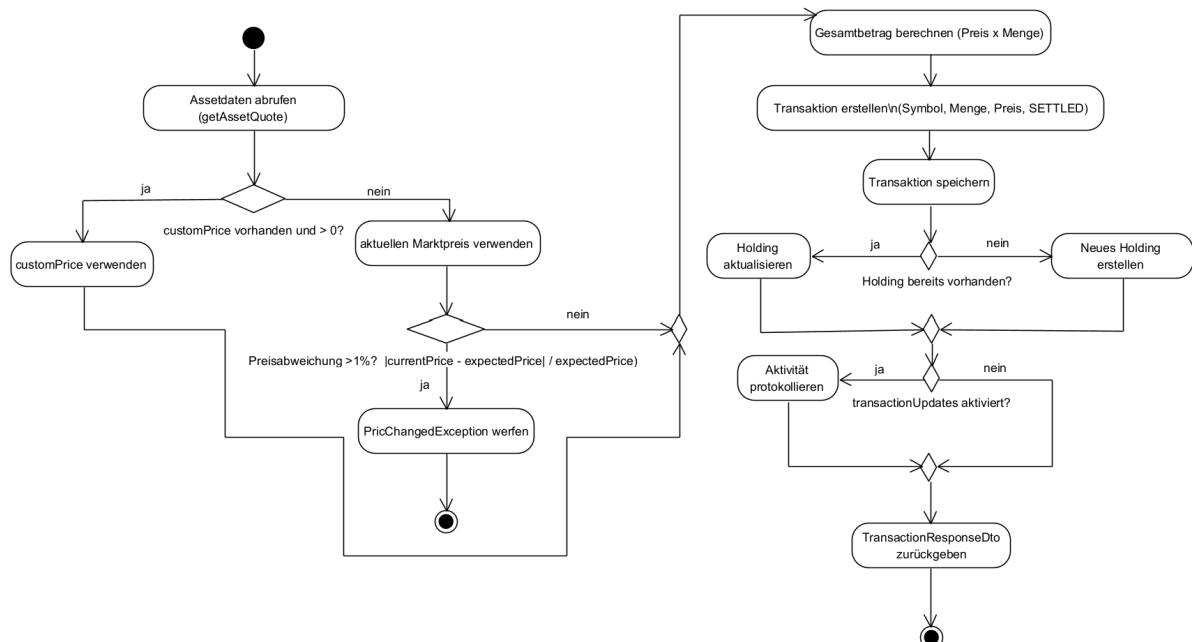


Abbildung 5.3: Ablauf eines Kaufprozesses

Nach der Preisprüfung berechnet der Service den Gesamtbetrag aus Preis und Menge. Anschließend wird eine Transaktion mit Typ BUY und Status SETTLED gespeichert. Danach wird der Bestand aktualisiert. Existiert noch kein Holding für Benutzer und Symbol, wird eines erstellt. Existiert bereits ein Holding, wird die Menge erhöht und die durchschnittliche Einstandsbasis neu berechnet. Diese Berechnung erfolgt als gewichteter Durchschnitt: Der alte Gesamtwert des Bestands und der neue Kaufwert werden addiert und durch die neue Gesamtmenge geteilt. Dadurch überschreibt ein späterer Kauf nicht den früheren Einstandskurs.

Beim Verkauf ist der Ablauf ähnlich, aber mit anderen Prüfungen. Zunächst muss ein Holding für das Symbol existieren. Danach wird geprüft, ob die vorhandene Menge ausreicht. Bei unzureichendem Bestand wird die Aktion abgelehnt. Anschliessend ruft das System den aktuellen Kurs ab, prüft die Preisabweichung, speichert eine Verkaufstransaktion und reduziert das Holding. Wenn die Restmenge null ist, wird das Holding gelöscht. Diese Logik sorgt dafür, dass aktuelle Bestandsdaten und historische Transaktionsdaten zusammenpassen.

Fachlich muss die Verarbeitung atomar bleiben: Eine gespeicherte Kauftransaktion ohne aktualisierten Bestand oder ein reduzierter Bestand ohne Verkaufstransaktion wäre ein inkonsistenter Zustand. Deshalb gehört die Logik in die Service-Schicht und nicht in einzelne Controller oder JavaScript-Funktionen. Fehlerfälle wie fehlende Holdings, ungültige Mengen und zu große Preisabweichungen werden vor oder während der Verarbeitung abgefangen. Nicht verfügbare Kursdaten werden über die Marktdaten-Fallbacks behandelt und führen je nach Situation zu neutralen Werten oder einer Rückmeldung an den Benutzer. Dadurch bleibt die Datenbank der verbindliche Zustand, während das Frontend nur Eingaben vorbereitet und Rückmeldungen anzeigt.

5.4 Portfolio- und Performanceberechnung

Die Portfolioansicht wird im Wesentlichen durch den PortfolioService erzeugt. Der Service sammelt die offenen Holdings eines Benutzers und reichert sie mit aktuellen Kursen, Marktwerten und Performancewerten an. Für die aktuelle Holding-Ansicht wird die Kostenbasis aus der im Holding gespeicherten durchschnittlichen Einstandsbasis und der offenen Menge gebildet. Für die Portfolio-Chartdaten wird zusätzlich die Transaktionshistorie ausgewertet, indem Käufe und Verkäufe zeitabhängig auf die gehaltene Menge und den investierten Betrag wirken.

Die Performance eines Holdings ergibt sich aus der Differenz zwischen aktuellem Wert und Kostenbasis beziehungsweise investiertem Betrag. Ist der investierte Betrag ungleich null, wird die Differenz durch den Betrag geteilt und in Prozent umgerechnet. Falls kein investierter Betrag vorhanden ist, aber ein positiver Marktwert besteht, wird ein Sonderfall behandelt. Diese Berechnung verhindert Divisionen durch null und macht die Darstellung stabiler. Die Werte werden gerundet, damit die Oberfläche nicht mit langen Nachkommastellen belastet wird.

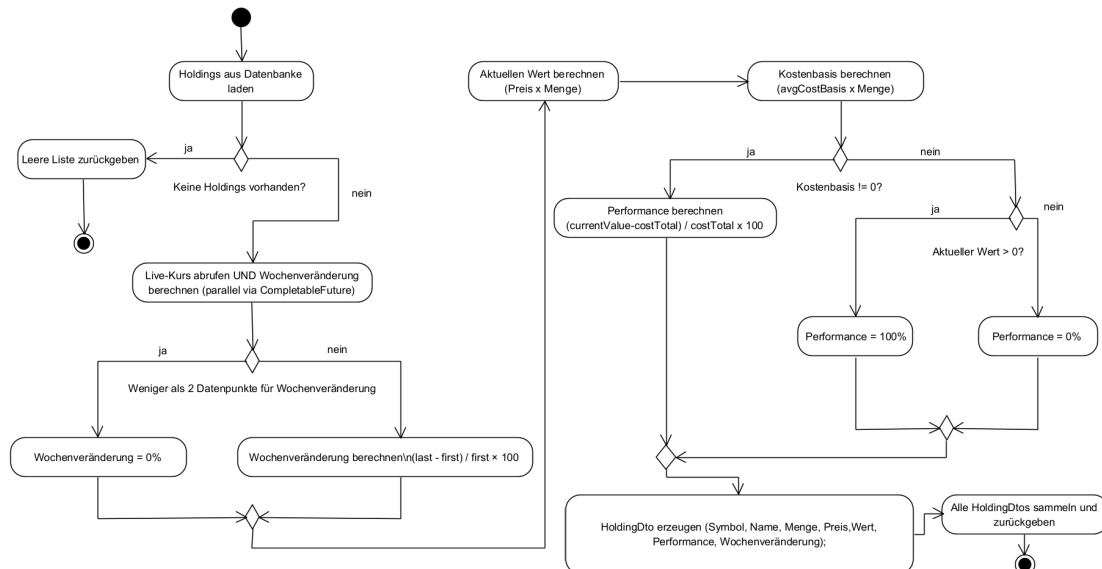


Abbildung 5.4: Darstellung der Berechnung von Holdings/Positionen

Der Algorithmus arbeitet damit in klaren Schritten: Zuerst werden die nutzerbezogenen Holdings geladen, danach werden pro Symbol aktuelle Marktdaten ergänzt. Aus Menge, durchschnittlicher Kostenbasis und aktuellem Kurs entstehen Marktwert, investierter Betrag, absolute Veränderung und prozentuale Rendite. Können externe Kursdaten nicht geladen werden, darf die gesamte Portfolioseite nicht ausfallen; stattdessen werden Fallback-Werte verwendet oder einzelne Kennzahlen leer beziehungsweise neutral dargestellt. Diese Fehlertoleranz ist bewusst gewählt, weil die Anwendung ein Dokumentationswerkzeug ist und auch bei gestörten Marktdaten noch die in der Anwendungsdatenbank gespeicherte Historie anzeigen soll.

Ein weiterer Aspekt ist die parallele Datenbeschaffung. Für jede Holding werden aktueller Kurs und wöchentliche Veränderung asynchron über `CompletableFuture` geladen. Diese Entscheidung folgt aus der Abhängigkeit von externen Marktdaten: Wenn mehrere Positionen vorhanden sind, würden rein sequenzielle Abrufe die Ladezeit erhöhen. Die parallele Verarbeitung verbessert die wahrgenommene Geschwindigkeit, ohne die fachliche Struktur zu verändern. Da die Anwendung dennoch serverseitig bleibt, wird die Komplexität einer vollständigen reaktiven Architektur vermieden. Die Portfolio-Chartdaten gehen noch weiter. Aus der Transaktionshistorie werden Zeitpunkte, gehaltene Mengen, historische Preise, Währungsumrechnung und Vergleichswerte kombiniert. Für verschiedene Zeiträume werden passende Yahoo-Intervalle gewählt, etwa Minuten-, Tages-, Wochen- oder Monatsdaten. Die Anwendung filtert Zeitstempel, ergänzt bei Bedarf einen aktuellen Endpunkt und berechnet pro Zeitpunkt die prozentuale Entwicklung des Portfolios. Dadurch kann der Benutzer nicht nur den aktuellen Zustand sehen, sondern auch eine zeitliche Entwicklung nachvollziehen. Die Berechnung verbindet damit historische Ereignisse und aktuelle Marktinformationen. Gerade diese Kombination begründet, warum Transaktionen dauerhaft gespeichert werden und Holdings nur den jeweils offenen Zustand zusammenfassen.

5.5 Benchmark-, EZB- und Inflationslogik

Neben der reinen Portfolioentwicklung bietet die Anwendung Vergleichswerte an. Dazu zählen ein EZB-bezogener Benchmark und Inflationsdaten. Diese Werte dienen nicht als vollständige finanzwissenschaftliche Analyse, sondern als orientierende Vergleichsgrößen. Der Benutzer soll sehen können, wie sich sein Portfolio im Vergleich zu Zins- oder Inflationsentwicklungen verhält. Diese Anforderung erweitert die reine Portfolioverwaltung um einen einfachen analytischen Kontext.

Die EZB-Logik wird im EcbService gekapselt. Der Service kann aktuelle und historische Einlagezinssätze bereitstellen. Im Portfoliochart wird für einen Zeitraum ein hypothetischer Zinsfaktor berechnet. Dafür wird zwischen Start- und Endzeitpunkt segmentweise der jeweils gültige Zinssatz verwendet. Die Zeitdauer eines Segments wird in Jahresbruchteile umgerechnet und mit einer Zinseszinsformel verarbeitet. Das Ergebnis wird als prozentuale Entwicklung ausgegeben. Diese Berechnung ist ein zentraler Algorithmus, weil sie historische Zinssätze zeitabhängig berücksichtigt.

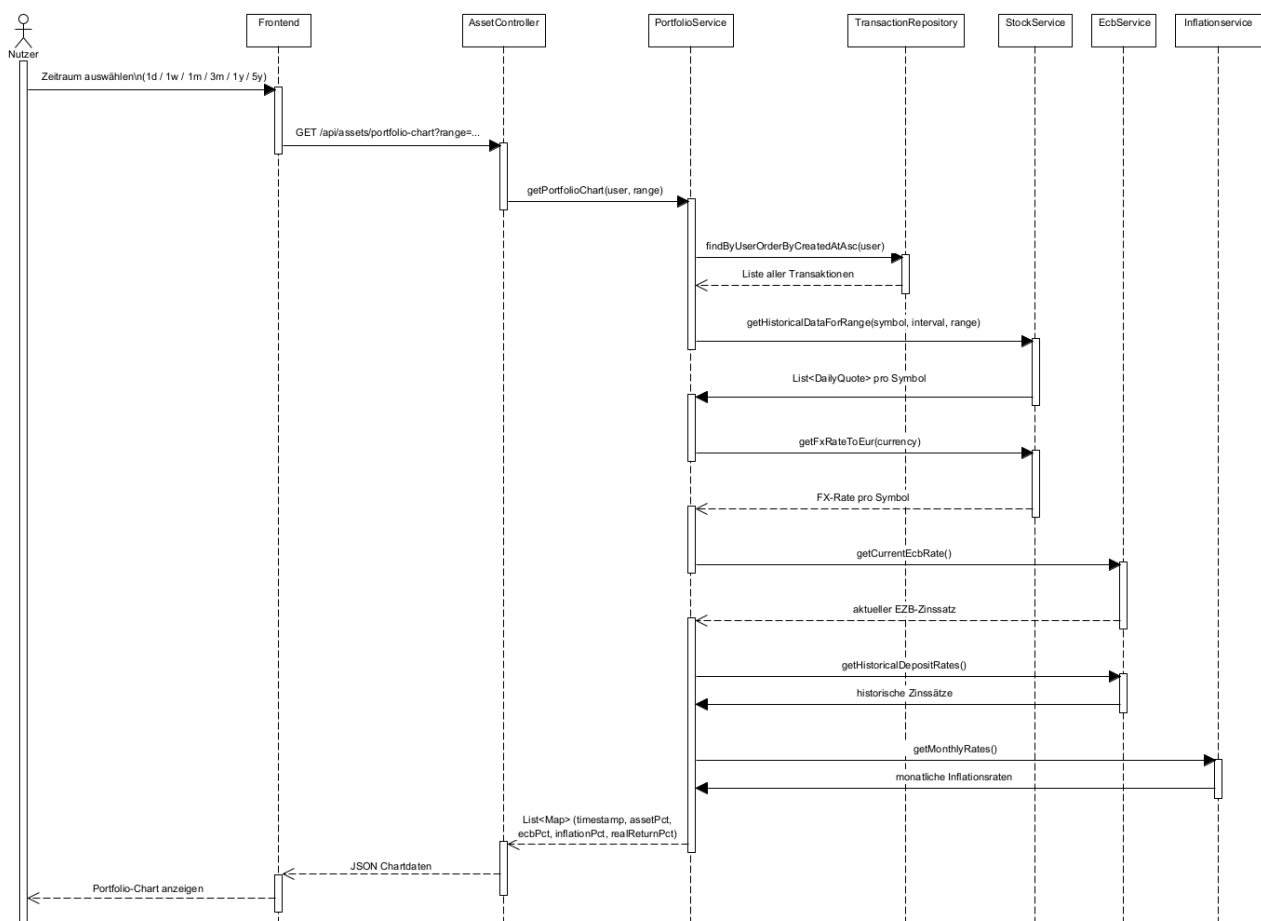


Abbildung 5.5: Erstellung der Chartdaten für das Portfolio

Die Inflationslogik wird über den `InflationService` eingebunden. Die Anwendung arbeitet mit deutschen monatlichen HICP-Raten von Eurostat, die für Chartpunkte passend zum Monat ausgewählt werden. Wenn für einen Monat kein Wert vorhanden ist, wird auf vorherige oder standardisierte Werte zurückgegriffen. Dadurch bleibt der Chart auch dann darstellbar, wenn externe Daten nicht vollständig sind. Die Darstellung als Kurve ist bewusst vereinfacht, weil die Projektarbeit den Fokus auf Anwendungsentwicklung legt und nicht auf eine vollständige volkswirtschaftliche Modellierung. Aus fachlicher Sicht sind diese Benchmarkwerte bewusst als Orientierung und nicht als Anlageberatung zu verstehen. Sie werden in die Auswertung integriert, beeinflussen aber weder Kauf- noch Verkaufslogik und verändern keine gespeicherten Portfolioereignisse. Damit bleibt die Anwendung ein Tracking- und Analysewerkzeug: Sie liefert nachvollziehbare Vergleichswerte, die Interpretation bleibt beim Benutzer.

5.6 Benachrichtigungen, Aktivitäten und Einstellungen

Benachrichtigungen und Aktivitäten ergänzen die Portfolioverwaltung um Nachvollziehbarkeit und Rückmeldung. Aktivitäten werden verwendet, um Ereignisse wie Käufe, Verkäufe oder Exporte in einer nutzerbezogenen Historie sichtbar zu machen. Dadurch kann der Benutzer später erkennen, welche Aktionen in der Anwendung ausgeführt wurden. Die Aktivitäten sind nicht identisch mit Transaktionen: Eine Transaktion ist ein fachliches Portfolioereignis, während eine Aktivität ein allgemeiner Anwendungseintrag ist. Diese Trennung verhindert Vermischung zwischen finanzieller Historie und Bedienhistorie.

Benachrichtigungen dienen als sichtbare Meldungen innerhalb der Anwendung. Sie besitzen Titel, Nachricht, Icon, Stil, Lesestatus und Erstellungszeitpunkt. Der Benutzer kann Benachrichtigungen abrufen, ungelesene zählen, als gelesen markieren oder löschen. Der `NotificationService` kapselt diese Funktionen. Die Benachrichtigungen werden beispielsweise beim Anwendungsstart aus Portfolio-Performancewerten oder bei bestimmten Systemereignissen erzeugt. Trade-Ereignisse werden dagegen als Aktivitäten protokolliert. Dadurch erhält der Benutzer eine zusätzliche Informationsschicht, ohne die eigentlichen Portfolioseiten ständig durchsuchen zu müssen.

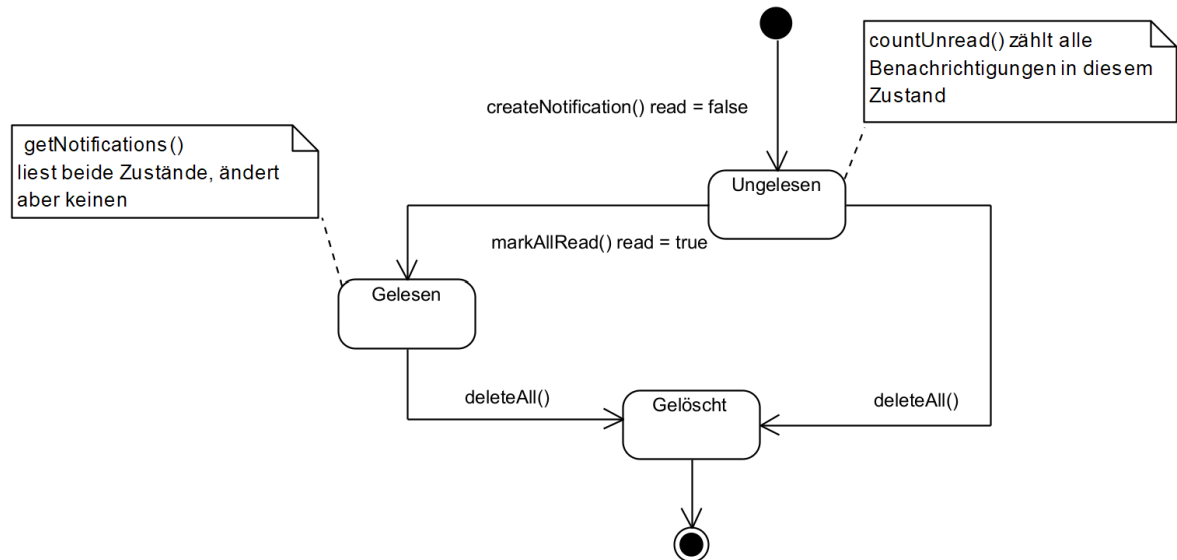


Abbildung 5.6: Benachrichtigung (Lebenszyklus: ungelesen → gelesen → gelöscht)

Die Einstellungen sind in `UserSettings` modelliert und gehören eins zu eins zu einem Benutzer. Sie umfassen aktuell Dark Mode, Preisalarme und Transaktionsupdates. Besonders die Einstellung für Transaktionsupdates beeinflusst die Aktivitätsprotokollierung: Wenn der Benutzer solche Updates deaktiviert, werden bestimmte Trade-Aktivitäten nicht protokolliert. Dadurch ist erkennbar, dass Einstellungen nicht nur dekorative UI-Daten sind, sondern fachliches Verhalten steuern können. Die Kombination aus Einstellungen, Aktivitäten und Benachrichtigungen verbessert die Benutzererfahrung, bleibt aber bewusst vom Portfolio-Kern getrennt. Portfolio- und Transaktionsdaten liegen in eigenen Tabellen, während Aktivitäten und Benachrichtigungen ergänzende Informationsobjekte sind. Dadurch können spätere Einstellungstypen, Benachrichtigungskategorien oder Aktivitätsfilter ergänzt werden, ohne die Grundlogik von Käufen, Verkäufen und Bestandsberechnung umzubauen.



6 Testkonzept und Validierung

Das Testkonzept leitet sich aus den Risiken der Anwendung ab. Kritisch sind vor allem Benutzerverwaltung, Transaktionslogik, Bestandsberechnung, Datenzugriff, Zugriffsschutz und UI-nahe Datenaufbereitung. Deshalb enthält das Projekt Java-Tests für Services, Controller, Repositories, Entities, DTOs und Security sowie JavaScript-Tests für Frontend-Hilfsfunktionen. Diese Mischung ist sinnvoll, weil die Anwendung sowohl serverseitige Fachlogik als auch clientseitige Interaktionen besitzt.

Repository-Tests prüfen, ob Datenbankzugriffe und abgeleitete Spring-Data-Methoden funktionieren. Dafür wird eine isolierte H2-In-Memory-Datenbank verwendet, damit keine echten Entwicklungs- oder Produktionsdaten verändert werden. Servicetests prüfen fachliche Regeln wie Benutzerregistrierung, Profilaktualisierung, Transaktionsverarbeitung, Portfoliowerte, Benachrichtigungen und externe Service-Fallbacks. Controllertests prüfen, ob Endpunkte erreichbar sind, passende Antworten liefern und Sicherheitsregeln respektieren. Security-Tests sind besonders wichtig, weil ungeschützte Portfolioendpunkte ein schwerer Fehler wären.

Tabelle 1: Testplanübersicht

Kategorie	Testklasse	Testart	Testschwerpunkte
Service	UserServiceTest	Unit (Mockito)	Registrierung, Login, Profilupdate, Passwort-Reset, Kontolöschung, Einstellungen
Service	TransactionServiceTest	Unit (Mockito)	Kaufen, Verkaufen, Preisabweichung >1%, Holding erstellen/aktualisieren
Service	PortfolioServiceTest	Unit (Mockito)	Holdings berechnen, Performance, Wochenveränderung
Service	StockServiceTest	Unit (Mockito)	Kursabruf, historische Daten, FX-Konvertierung, Fallback bei API-Ausfall
Service	NotificationServiceTest	Unit (Mockito)	Benachrichtigung erstellen, lesen, löschen, Performance-Alerts
Service	ActivityServiceTest	Unit (Mockito)	Aktivität protokollieren, Verlauf abrufen
Service	EcbServiceTest	Unit (Mockito)	EZB-Zinssatz abrufen, historische Raten
Service	InflationServiceTest	Unit (Mockito)	Monatliche Inflationsraten, Fallback
Controller	AssetControllerTest	Unit (MockMvc)	REST-Endpunkte, Kauf/-Verkauf, Preisbestätigung 409
Controller	AuthControllerTest	Unit (MockMvc)	Login-Seite, Registrierung, Profil-API
Controller	StockControllerTest	Unit (MockMvc)	Kursendpunkte, Historien-API, FX-Endpunkt
Controller	NotificationControllerTest	Unit (MockMvc)	Benachrichtigungen abrufen, markieren, löschen
Controller	ActivityControllerTest	Unit (MockMvc)	Aktivitätsverlauf-Endpunkt
Security	SecurityConfigTest	Integration (SpringBootTest)	Öffentliche Endpunkte, geschützte Seiten, Redirect ohne Authentifizierung
Repository	UserRepositoryTest	Integration (H2)	findByEmail, existsByEmail
Repository	TransactionRepositoryTest	Integration (H2)	findByUser, findBySymbol
Repository	HoldingRepositoryTest	Integration (H2)	findByUserAndSymbol, deleteByUser
Repository	NotificationRepositoryTest	Integration (H2)	countUnread, markAllRead, deleteByUser
Repository	ActivityRepositoryTest	Integration (H2)	findByUser, clearActivities
Repository	UserSettingsRepositoryTest	Integration (H2)	findByUser, Standardwerte
Entity/DTO	UserTest	Unit	Feldbelegung, Validierung
Entity/DTO	UserRegistrationDtoTest	Unit	Passwortabgleich, Validierungsregeln
Entity/DTO	UserProfileUpdateRequestDtoTest	Unit	Feldvalidierung
Exception	PriceChangedExceptionTest	Unit	Fehlermeldung, aktueller Preis im Exception-Objekt

Testrahmen-Übersicht

Testart	Framework	Datenbank
Unit Tests (Service/Controller)	JUnit 5 + Mockito	keine (gemockt)
Integrationstests (Repository)	JUnit 5 + Spring	H2 In-Memory
Security Tests	SpringBootTest + MockMvc	H2 In-Memory
JavaScript-Tests	Jest + jsdom	keine

Die Testbereiche decken die größten fachlichen Risiken gezielt ab. Die Service-Tests prüfen, ob Käufe, Verkäufe und Performancewerte auch ohne echte externe Dienste korrekt verarbeitet werden. Repository-Tests sichern die nutzerbezogene Datenhaltung ab, während Security-Tests verhindern, dass geschützte Seiten oder API-Endpunkte ohne Anmeldung erreichbar sind. Damit validiert das Testkonzept nicht jede mögliche Darstellung der Oberfläche vollständig, sondern vor allem die Teile, deren Fehler direkte Auswirkungen auf Datenintegrität, Datenschutz oder Portfolioauswertung hätten.

Die JavaScript-Tests konzentrieren sich auf UI-Logik, die ohne echten Browser über Jest und jsdom geprüft werden kann. Dazu gehören Formatierung von Mengen und Preisen, HTML-Escaping, Suchtexte für Transaktionen, Tabellenfilter, Pagination, Dashboard-Zusammenfassungen, Chartaufbereitung, CSV-Escaping und CSRF-Header-Erzeugung. Besonders HTML-Escaping ist relevant, weil Benutzereingaben oder externe Namen nicht ungeprüft in Tabellen gerendert werden sollten. Die Tests tragen damit nicht nur zur Korrektheit, sondern auch zur Sicherheit bei.

Als Abnahmetests eignen sich szenariobasierte Prüfungen. Ein Benutzer registriert sich, meldet sich an, sucht ein Asset, kauft eine Menge, sieht das Holding im Portfolio, verkauft einen Teil, prüft die reduzierte Menge und kontrolliert die Transaktionshistorie. Ein weiterer Abnahmetest prüft Preisabweichungen: Wenn der erwartete Preis deutlich vom aktuellen Preis abweicht, muss die Aktion abgelehnt werden. Ein dritter Test prüft Zugriffsschutz: Ohne Login dürfen geschützte Seiten nicht erreichbar sein. Diese Tests validieren direkt die wichtigsten Anforderungen.

7 Technische Inbetriebnahme und Nutzungseinführung

7.1 Technische Inbetriebnahme

Die technische Inbetriebnahme folgt aus der gewählten Architektur. Die Anwendung benötigt eine Java-17-Laufzeit, Maven beziehungsweise den Maven Wrapper, Node.js für JavaScript-Tests und eine PostgreSQL-Datenbank. Für Entwicklung und Abgabe ist Neon als Cloud-PostgreSQL vorgesehen, alternativ kann lokal PostgreSQL genutzt werden. Die Datenbankverbindung wird über Umgebungsvariablen konfiguriert. Dazu gehören `SPRING_PROFILES_ACTIVE`, `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD` und `SPRING_JPA_HIBERNATE_DDL_AUTO`.

Der Start erfolgt im einfachsten Fall über den Maven Wrapper mit `./mvnw spring-boot:run` beziehungsweise unter `mvn spring-boot:run`. Alternativ können die vorhandenen Startskripte genutzt werden: `start-dev.bat` baut die Anwendung neu und startet anschließend die JAR-Datei, `start.bat` startet nur eine bereits gebaute JAR-Datei. Die JavaScript-Testabhängigkeiten werden über `npm install` installiert, anschließend können die Frontend-Tests mit `npm run test:js` oder `npm.cmd run test:js` ausgeführt werden. Ein produktionsnaher Build kann mit `mvn clean package` erzeugt werden.

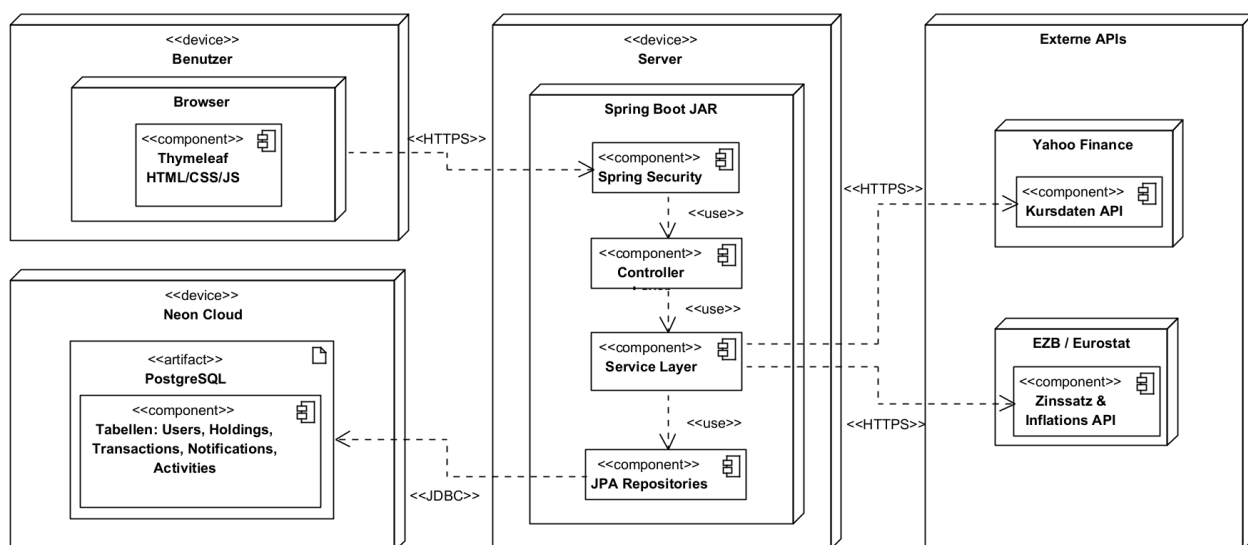


Abbildung 7.1: Laufzeitarchitektur (Browser, Spring-Boot, Neon, externe APIs)

Die Anwendung liest die Konfiguration aus `application.yaml` und profilspezifischen Dateien. Das Entwicklungsprofil kann SQL-Logging und Template-Neuladen erleichtern. Das Produktionsprofil sollte stabilere Einstellungen verwenden. Für die Datenbank-Schema-Erzeugung ist im Projekt `ddl-auto` konfigurierbar. Zu Demozwecken ist `update` aktiv, während `create-drop` nur für kurzlebige Entwicklungsumgebungen geeignet wäre, da Daten beim Neustart verloren gehen würden. Diese Entscheidung sollte vor der Endkunden-Inbetriebnahme bewusst noch getroffen werden.

7.2 Schulungsunterlage: Migration von Neon zu einer eigenen PostgreSQL-Datenbank

Diese Schulung beschreibt den Wechsel von der vorkonfigurierten Neon-Datenbank auf eine selbst betriebene PostgreSQL-Datenbank. Ziel ist, dass die Anwendung nach der Migration mit denselben fachlichen Daten läuft, aber nicht mehr auf den externen Neon-Dienst angewiesen ist. Der Ablauf ist bewusst als kontrollierte Migration aufgebaut: Zuerst wird der aktuelle Datenbestand gesichert, danach wird die Ziel-Datenbank vorbereitet, anschließend werden Daten importiert und erst zum Schluss wird die Anwendung umkonfiguriert.

Lernziel: Die Teilnehmenden können eine bestehende Stock-Tracker-Installation von Neon auf eine eigene PostgreSQL-Instanz umstellen, die Verbindung über Umgebungsvariablen konfigurieren und die erfolgreiche Migration fachlich prüfen.

Voraussetzungen: Benötigt werden Zugriff auf die Neon-Verbindungsdaten, Zugriff auf die eigene PostgreSQL-Instanz, die PostgreSQL-Clientwerkzeuge `pg_dump`, `pg_restore`, `createdb` und `psql` sowie ein Benutzer mit Rechten zum Anlegen und Beschreiben der Ziel-Datenbank. Die Anwendung sollte während des Exports nicht produktiv genutzt werden, damit kein Datenstand zwischen Export und Umschaltung verloren geht.

1. **Quell- und Zielverbindung erfassen.** Die Neon-URL wird aus der bisherigen Konfiguration übernommen. Für die eigene PostgreSQL-Instanz werden Host, Port, Datenbankname, Benutzer und Passwort festgelegt. Zugangsdaten werden nicht in die Versionsverwaltung geschrieben, sondern nur als lokale Umgebungsvariablen oder Server-Secrets gesetzt.
2. **Neon-Datenbestand sichern.** Der Export sollte im Custom-Format erfolgen, weil er mit `pg_restore` gezielt wiederhergestellt werden kann.

```
1 pg_dump
   "postgres://NEON_USER:NEON_PASS@NEON_HOST/neondb?sslmode=require"
   \
2 --format=custom \
3 --file=stock_tracker_neon.backup
```

3. **Eigene PostgreSQL-Datenbank vorbereiten.** Auf der Zielinstanz wird eine neue Datenbank angelegt. Der Datenbankbenutzer erhält die notwendigen Rechte für Tabellen, Sequenzen und Inserts.

```
1 createdb -h localhost -p 5432 -U postgres stock_tracker
```

4. **Dump in die Ziel-Datenbank einspielen.** Der Restore wird gegen die neue Datenbank ausgeführt. Bei einer leeren Ziel-Datenbank reicht ein normaler Import; bei einer wiederverwendeten Datenbank sollte vorher geklärt werden, ob bestehende Tabellen gelöscht oder überschrieben werden dürfen.

```
1 pg_restore -h localhost -p 5432 -U postgres \  
2   --dbname=stock_tracker \  
3   --verbose \  
4   stock_tracker_neon.backup
```

5. **Anwendung auf die neue Datenbank umstellen.** Die Anwendung wird nicht im Quellcode geändert, sondern über Umgebungsvariablen konfiguriert. Für Windows PowerShell kann die Schulungsumgebung beispielsweise so gesetzt werden:

```
1 $env:SPRING_PROFILES_ACTIVE="prod"  
2 $env:SPRING_DATASOURCE_URL="jdbc:postgresql://localhost:5432/stock_tracker"  
3 $env:SPRING_DATASOURCE_USERNAME="postgres"  
4 $env:SPRING_DATASOURCE_PASSWORD="postgres"  
5 $env:SPRING_JPA_HIBERNATE_DDL_AUTO="validate"
```

Für eine Demo kann statt `validate` vorübergehend `update` verwendet werden, wenn Hibernate fehlende Schema-Änderungen ergänzen soll. Für einen stabilen Betrieb ist `validate` besser, weil die Anwendung dann nicht unbemerkt Strukturen in der Datenbank verändern kann.

6. **Anwendung starten und Migration prüfen.** Nach dem Start werden Login, Dashboard, Portfolio, Transaktionen, Kauf, Verkauf und Export geprüft. Zusätzlich sollte in den Logs kontrolliert werden, ob die JDBC-Verbindung auf die neue PostgreSQL-Instanz zeigt und keine Neon-Hostnamen mehr verwendet werden.

Abnahmekriterien der Migration: Die Anwendung startet ohne Datenbankfehler, vorhandene Benutzer können sich anmelden, bestehende Holdings und Transaktionen sind sichtbar, neue Transaktionen können gespeichert werden und ein Neustart der Anwendung erhält die Daten. Damit ist nachgewiesen, dass nicht nur die Verbindung technisch funktioniert, sondern auch die fachlich relevanten Daten korrekt übernommen wurden.

Rollback: Bis die Prüfung abgeschlossen ist, bleibt die Neon-Datenbank unverändert bestehen. Falls der Import oder die Anwendung gegen die eigene PostgreSQL-Datenbank fehlschlägt, werden die Umgebungsvariablen wieder auf die Neon-Verbindung gesetzt und die Anwendung erneut gestartet. Erst nach erfolgreicher Abnahme sollte Neon deaktiviert oder gelöscht werden.

Ein Inbetriebnahmeablauf sollte deshalb mindestens folgende Schritte enthalten: Repository bereitstellen, Umgebungsvariablen setzen, Datenbankverbindung prüfen, Tests ausführen, Anwendung bauen, Anwendung starten, Login und Registrierung prüfen, Asset-Suche prüfen, Kauf- und Verkaufsszenario testen und Logs auf Fehler kontrollieren. Bei externer Bereitstellung müssen zusätzlich HTTPS, sichere Umgebungsvariablen und gegebenenfalls Forwarded-Header-Konfiguration beachtet werden. Damit ist die Inbetriebnahme nicht nur ein technischer Startbefehl, sondern ein kontrollierter Ablauf zur Sicherstellung der Betriebsbereitschaft.

7.3 Einführung der Nutzung

Diese Nutzungseinführung richtet sich an Endnutzer, die ihre Portfolioverwaltung in der Anwendung beginnen möchten. Der Einstieg erfolgt über Registrierung, Login und Dashboard. Von dort wechselt der Benutzer zur Portfolioseite, sucht über den Add-Asset-Dialog ein Asset, prüft die angezeigten Kursinformationen und erfasst Menge sowie gegebenenfalls einen individuellen Kaufpreis.

Nach dem Speichern eines Kaufs erscheint die Position im Portfolio mit Menge, durchschnittlicher Kostenbasis, aktuellem Preis, aktuellem Wert und Performance. Weitere Käufe desselben Symbols erhöhen die Menge und aktualisieren den gewichteten durchschnittlichen Einstandspreis. Verkäufe werden aus der Portfolioansicht gestartet; die Anwendung prüft dabei den vorhandenen Bestand und verhindert Überverkäufe. Die Transaktionsseite dient anschließend als Historie und kann gefiltert oder exportiert werden.

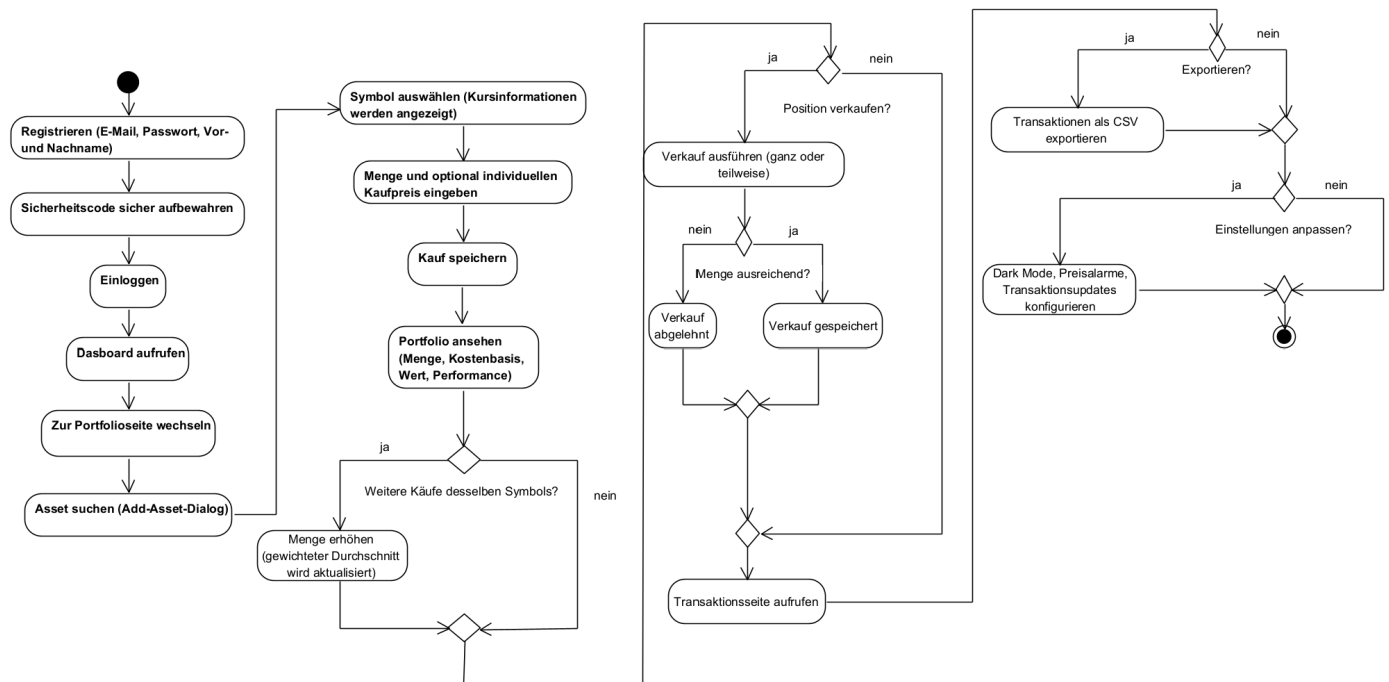


Abbildung 7.2: Benutzerfluss: Registrierung, Portfolioverwaltung, Käufe/Verkäufe, Exporte und Einstellungen

Die Anwendung führt keine echten Börsenorders aus, sondern dokumentiert/simuliert ausschließlich Portfolio-Ereignisse. Typische Fehlerfälle: falscher Login, fehlende Kursdaten, leeres Portfolio, abgelehnter Verkauf wegen unzureichender Menge und abgelehnter Kauf wegen Preisveränderung. Diese Fehlerfälle sind durch eine erneute Eingabe/Suche mit korrekten Werten zu lösen.

8 Fazit und Ausblick

Zusammenfassend erfüllt der Stock Tracker die zentrale Zielsetzung einer funktionsfähigen, webbasierten Portfolioverwaltung. Besonders prägend sind die nutzerbezogene Datenhaltung, die Trennung von aktuellem Bestand und Transaktionshistorie, die abgesicherte Kauf- und Verkaufslogik sowie die Kombination eigener Portfoliodaten mit externen Marktdaten. Dadurch entsteht keine Handelsplattform, sondern ein nachvollziehbares Dokumentations- und Analysewerkzeug.

Die fachlichen Arbeitsschritte bauen kausal aufeinander auf: Aus der Problemstellung entstehen Anforderungen, daraus Anwendungsfälle, daraus UI, Datenmodell, Architektur und zentrale Algorithmen. Die Tests und die Inbetriebnahmebeschreibung schließen diesen Weg ab, weil sie zeigen, wie die geforderte Funktionalität validiert und praktisch bereitgestellt werden kann.

Literatur

- [1] Helmut Balzert. Lehrbuch der Softwaretechnik: Entwurf, Implementierung, Installation und Betrieb. Spektrum Akademischer Verlag, 3 edition, 2012. ISBN 978-3-8274-1706-0.
- [2] European Central Bank. ECB Data Portal – Statistical Data Warehouse. REST API, 2024. URL <https://data.ecb.europa.eu/help/api/data>. Historische Einlagezinssätze für den EZB-Benchmark.
- [3] Eurostat. HICP – Harmonisierter Verbraucherpreisindex. Statistische Datenbank der Europäischen Kommission, 2024. URL <https://ec.europa.eu/eurostat/web/hicp/data/database>. Monatliche HICP-Raten für Deutschland (Inflationsvergleich im Portfoliochart).
- [4] Meta Platforms, Inc. Jest – Delightful JavaScript Testing, 2024. URL <https://jestjs.io/docs/getting-started>.
- [5] Mockito Contributors. Mockito – mocking framework for unit tests in Java, 2024. URL <https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html>.
- [6] Neon, Inc. Neon – Serverless Postgres. Cloud-Datenbankdienst, 2024. URL <https://neon.tech/docs/introduction>.
- [7] OWASP Foundation. OWASP Top Ten. Sicherheitsrichtlinie, 2021. URL <https://owasp.org/www-project-top-ten/>.
- [8] OWASP Foundation. Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet. OWASP Cheat Sheet Series, 2024. URL https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html.
- [9] The Thymeleaf Team. Thymeleaf – Using Thymeleaf, 2023. URL <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>.
- [10] VMware, Inc. Spring Boot Reference Documentation, 2024. URL <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Version 3.x.
- [11] VMware, Inc. Spring Data JPA Reference Documentation, 2024. URL <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>.
- [12] VMware, Inc. Web on Servlet Stack – Spring MVC Reference, 2024. URL <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html>.
- [13] VMware, Inc. Spring Security Reference Documentation, 2024. URL <https://docs.spring.io/spring-security/reference/>. Version 6.x.
- [14] Yahoo Inc. Yahoo Finance. Marktdatenportal, 2024. URL <https://finance.yahoo.com/>. Kursabfragen, historische Preisreihen und Asset-Suche.